Sieci Neuronowe – projekt 1

Perceptron wielowarstwowy w zadaniach klasyfikacji i regresji

Jakub Pietrzak, Filip Filipkowski

Październik 2026

Streszczenie

W projekcie zaimplementowano od podstaw perceptron wielowarstwowy (z użyciem wyłącznie biblioteki numpy), uczony algorytmem propagacji wstecznej błędu i metodą SGD, wraz z wizualizacją wyników, przebiegu błędu i ewolucji wag. Sieć zbadano w zadaniach regresji funkcji $\mathbb{R} \to \mathbb{R}$ (zbiory `cube` i `activation`) oraz klasyfikacji punktów w $\mathbb{R}^2$ (zbiory `simple` i `three_gauss`). Sprawdzono wpływ funkcji aktywacji (Sigmoid, Tanh, ReLU), liczby warstw ukrytych (od 0 do 4) i ich liczności oraz funkcji błędu (MSE i BCE w regresji, CCE, BCE i MSE w klasyfikacji). W regresji najlepiej sprawdziła się funkcja Tanh i jedna warstwa ukryta z 16–64 neuronami. Kolejne warstwy nie poprawiały wyników, a wyjście sigmoidalne uniemożliwiało przewidywanie wartości spoza zakresu uczącego. Żadna sieć nie potrafiła poprawnie ekstrapolować funkcji sześciennej. W klasyfikacji wszystkie architektury z co najmniej dwoma neuronami ukrytymi osiągnęły 92–94% dokładności na zbiorze `three_gauss`. Wynik ten ogranicza nakładanie się klas, a wybór funkcji aktywacji i funkcji błędu zmieniał go o mniej niż 1 punkt procentowy. W drugiej części ta sama implementacja, w konfiguracji 784-256-128-10 z ReLU i malejącym krokiem uczenia, osiągnęła 98,42% dokładności na zbiorze testowym MNIST.

Spis treści

1 Wstęp **3**

2 Opis sieci i implementacji **3**
2.1 Model perceptronu wielowarstwowego . . . 3
2.2 Inicjalizacja wag i powtarzalność . . . 3
2.3 Struktura programu . . . 3
2.4 Wizualizacje . . . 4

3 Zbiory danych **4**

4 Metodyka eksperymentów **4**

5 Wyniki – regresja **5**
5.1 Zbiory danych i sposób oceny . . . 5
5.2 Przebieg eksperymentów . . . 6
5.3 Wpływ funkcji aktywacji . . . 6
5.4 Wpływ liczby warstw ukrytych i ich liczności . . . 9
5.5 Wpływ funkcji błędu . . . 15
5.6 Ewolucja wag . . . 17
5.7 Powtarzalność wyników . . . 18
5.8 Podsumowanie wyników regresji . . . 18

6 Wyniki – klasyfikacja **18**
6.1 Zbiory danych i sposób oceny 18
6.2 Przebieg eksperymentów 19
6.3 Sieć bez warstwy ukrytej 20
6.4 Wpływ liczby warstw ukrytych i ich liczności 22
6.5 Wpływ funkcji aktywacji 25
6.6 Wpływ funkcji błędu 26
6.7 Powtarzalność wyników 26
6.8 Podsumowanie wyników klasyfikacji 27

7 Część druga – rozpoznawanie cyfr MNIST **27**
7.1 Opis problemu i zbiór danych 27
7.2 Architektura sieci i parametry uczenia 28
7.3 Przebieg eksperymentów 28
7.4 Wyniki 29
7.5 Najlepsza sieć i odtworzenie uczenia 30
7.6 Weryfikacja na własnoręcznie narysowanych cyfrach 31
7.7 Wnioski z części drugiej 32

8 Podsumowanie i wnioski **33**

1 Wstęp

Perceptron wielowarstwowy (MLP) to sieć neuronowa złożona z warstw neuronów, w której każdy neuron oblicza ważoną sumę wyjść poprzedniej warstwy i przepuszcza ją przez nieliniową funkcję aktywacji. Sieć uczy się w sposób nadzorowany: na podstawie par (wejście, oczekiwane wyjście) dobiera wagi tak, aby zminimalizować funkcję błędu. Wykorzystuje do tego algorytm propagacji wstecznej błędu [1, 2].

Celem projektu była własna implementacja MLP (bez bibliotek sieci neuronowych, wyłącznie z użyciem numpy [6]) i zbadanie jej działania w dwóch zadaniach:

- **regresji** – przybliżenia funkcji $f : \mathbb{R} \to \mathbb{R}$ na podstawie próbek,
- **klasyfikacji** – przypisania punktu z $\mathbb{R}^2$ do jednej z kilku klas.

Zbadano wpływ funkcji aktywacji, liczby i liczności warstw ukrytych oraz funkcji błędu na skuteczność sieci. W drugiej części tą samą implementację zastosowano do rozpoznawania ręcznie pisanych cyfr ze zbioru MNIST [5].

Rozdział 2 opisuje sieć i program, rozdziały 3 i 4 dane i sposób prowadzenia eksperymentów. Wyniki przedstawiono w rozdziałach 5 (regresja), 6 (klasyfikacja) i 7 (MNIST), a wnioski w rozdziale 8.

2 Opis sieci i implementacji

2.1 Model perceptronu wielowarstwowego

Wyjście warstwy l jest obliczane jako

$$a^{(l)} = \sigma\Big(W^{(l)} a^{(l-1)} + b^{(l)}\Big), \tag{1}$$

gdzie $W^{(l)}$ to macierz wag, $b^{(l)}$ wektor biasów, a σ funkcja aktywacji. Gradient funkcji błędu L względem wag obliczany jest propagacją wsteczną, a wagi aktualizowane metodą stochastycznego spadku gradientu (SGD) na paczkach danych:

$$W^{(l)} \leftarrow W^{(l)} - \eta \, \frac{\partial L}{\partial W^{(l)}}, \tag{2}$$

gdzie η to krok uczenia. Zaimplementowano funkcje aktywacji Sigmoid, Tanh, ReLU, liniową i Softmax, funkcje błędu MSE, BCE i CCE oraz opcjonalną zmianę kroku uczenia między epokami (m.in. spadek wykładniczy).

2.2 Inicjalizacja wag i powtarzalność

Wagi są losowane z rozkładu $\mathcal{N}(0,\, 1/n_{\text{in}})$, gdzie n_{in} to liczba wejść neuronu (inicjalizacja LeCuna [4]), a biasy z rozkładu $U[0, 1)$. Do inicjalizacji i do tasowania danych w każdej epoce używany jest jeden generator `numpy.random.default_rng(seed)`, więc to samo ziarno daje identyczny przebieg uczenia.

2.3 Struktura programu

Rdzeń sieci znajduje się w katalogu `nn/`: klasa `MLP`, funkcje aktywacji, funkcje błędu i harmonogramy kroku uczenia. Eksperymenty definiuje się w pliku `configs.py`, podając liczbę i rozmiar warstw, funkcje aktywacji, funkcję błędu, krok uczenia, liczbę epok, rozmiar paczki i ziarna. Uruchamia się je poleceniem `python main.py run <nazwa>`. Wyniki (tabele, wykresy, modele) zapisywane są automatycznie. Poprawność propagacji w przód i wstecz sprawdzono na przykładzie z wykładu (`python main.py test`).

2.4 Wizualizacje

Program tworzy następujące wizualizacje:

- **wyniki uczenia** – dla regresji krzywa przewidywana przez sieć na tle funkcji docelowej, dla klasyfikacji obszary decyzyjne na tle punktów testowych,
- **przebieg uczenia** – błąd uczenia oraz dokładność lub MSE w kolejnych epokach,
- **ewolucja wag** – interaktywny plik `network.html`, w którym suwakiem przechodzi się przez kolejne epoki (grubość i kolor krawędzi odpowiadają wartości wagi, wielkość neuronu wartości biasu).

3 Zbiory danych

Wykorzystano cztery zbiory syntetyczne (tabela 1), każdy w wariantach o 100, 500, 1000 i 10 000 punktach. Szczegółowe opisy i rysunki zbiorów znajdują się na początku rozdziałów 5 i 6 (rysunki 1 i 15).

Tabela 1: Zbiory danych użyte w eksperymentach.

Zbiór	Zadanie	Opis
`simple`	klasyfikacja, 2 klasy	punkty w $[-1, 1]^2$ rozdzielone prostą (liniowo separowalne)
`three_gauss`	klasyfikacja, 3 klasy	trzy nachodzące na siebie rozkłady normalne
`activation`	regresja	funkcja w kształcie „schodka” (podobna do funkcji sigmoidalnej)
`cube`	regresja	wielomian trzeciego stopnia

W zbiorach regresyjnych punkty uczące pochodzą z przedziału $x \in [-1{,}9;\ 5]$, a testowe z szerszego przedziału $[-5;\ 7]$. Pozwala to ocenić zarówno interpolację, jak i ekstrapolację. W zbiorach klasyfikacyjnych zbiór testowy ma ten sam rozkład co uczący.

4 Metodyka eksperymentów

W każdym eksperymencie zmieniano jeden parametr sieci, a pozostałe miały wartości z konfiguracji bazowej:

- **regresja** – zbiór `cube_100`, sieć 1-16-1 z Tanh, wyjście liniowe, MSE, $\eta = 0{,}05$, 500 epok, paczka rozmiaru 1,
- **klasyfikacja** – zbiór `three_gauss_1000`, sieć 2-8-8-3 z funkcją sigmoidalną, wyjście Softmax, CCE, $\eta = 0{,}1$, 20 epok, paczka rozmiaru 1.

Jakość regresji mierzono błędami MSE i MAE na zakresie uczącym oraz na całym zbiorze testowym, a jakość klasyfikacji dokładnością na zbiorze testowym. Większość przebiegów wykonano z ziarnem 42. Eksperymenty z funkcją błędu oraz dodatkową serię klasyfikacyjną powtórzono dla ziaren 1, 2 i 3 i podano średnią $\pm$ odchylenie standardowe. Na tej podstawie oceniono, które różnice między wariantami są większe od zmienności wynikającej z losowej inicjalizacji wag.

5 Wyniki – regresja

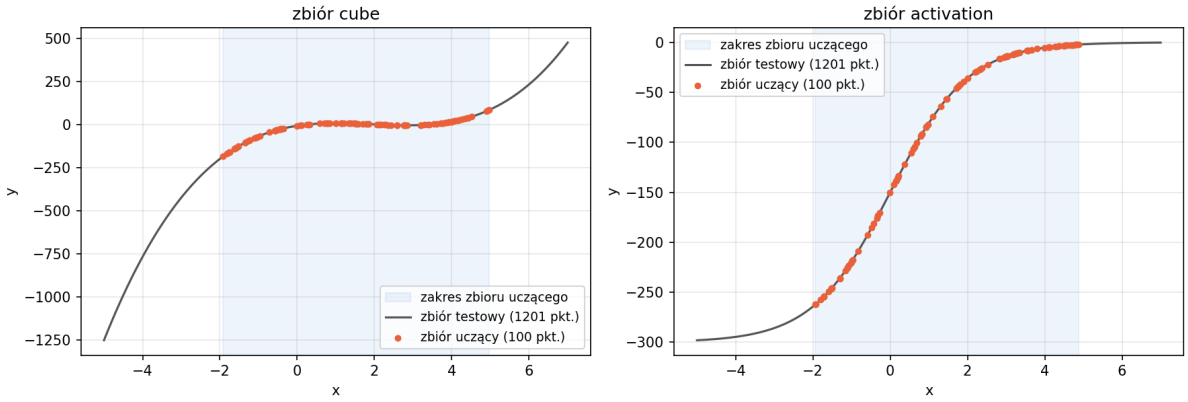
5.1 Zbiory danych i sposób oceny

Zadanie regresji polega na nauczeniu sieci funkcji $f : \mathbb{R} \to \mathbb{R}$ na podstawie skończonej liczby punktów (x, y). Wykorzystano dwa zbiory (rysunek 1):

- `cube` – wielomian trzeciego stopnia, który w zakresie uczącym jest prawie płaski, a poza nim szybko rośnie (dla $x > 5$) i maleje (dla $x < -2$),
- `activation` – funkcja w kształcie „schodka” (przeskalowana funkcja sigmoidalna), która poza zakresem uczącym jest prawie stała.

W obu przypadkach zbiór uczący zawiera 100 losowo rozmieszczonych punktów z przedziału ok. $x \in [-1{,}9;\ 5{,}0]$, a zbiór testowy 1201 równomiernie rozmieszczonych punktów z szerszego przedziału $x \in [-5;\ 7]$. Zbiór testowy sprawdza więc dwie rzeczy: interpolację (czy sieć dobrze odtwarza funkcję tam, gdzie miała dane) i ekstrapolację (jak sieć „przedłuża” funkcję poza zakres danych uczących).

Oba zbiory różnią się pod względem trudności ekstrapolacji. Funkcja sześcienna rośnie znacznie szybciej niż jakakolwiek funkcja liniowa, a sieć z aktywacjami Tanh lub Sigmoid poza zakresem uczącym dąży do stałej (neurony się nasycają), z ReLU zaś rośnie co najwyżej liniowo. Na podstawie danych z przedziału, w którym funkcja jest prawie płaska, sieć nie ma więc możliwości odtworzenia jej gwałtownego wzrostu na brzegach, dlatego ekstrapolacja zbioru `cube` jest z założenia bardzo trudna. Funkcja `activation` wypłaszcza się natomiast na brzegach dokładnie tak jak funkcje Tanh i Sigmoid, więc nasycenie neuronów ukrytych daje w tym przypadku poprawną ekstrapolację.



Rysunek 1: Zbiory regresyjne: punkty uczące (100 punktów), zbiór testowy (1201 punktów) i zakres zbioru uczącego (zaznaczony na niebiesko).

Skalowanie danych. Przed uczeniem wartości x są standaryzowane (średnia 0, odchylenie standardowe 1), a wartości y są przeskalowywane do przedziału $[0, 1]$. Statystyki skalowania (średnią, odchylenie, minimum i maksimum) liczy się wyłącznie na zbiorze uczącym i tymi samymi wartościami przekształca się zbiór testowy. Dlatego wartości y zbioru testowego wychodzą poza $[0, 1]$: dla zbioru `cube` mieszczą się w przedziale ok. $[-4;\ 2{,}4]$.

Miary jakości. Jakość dopasowania mierzono błędem średniokwadratowym (MSE) i średnim błędem bezwzględnym (MAE), liczonymi na przeskalowanych wartościach y. Obie miary podano dla dwóch podzbiorów zbioru testowego:

- **zakres uczący** – 688 punktów testowych, których x leży w przedziale zbioru uczącego (miara jakości interpolacji),

- **cały zbiór testowy** – wszystkie 1201 punktów (obejmuje także ekstrapolację).

Konfiguracja bazowa. Jeśli nie zaznaczono inaczej, używano sieci 1-16-1 (jedna warstwa ukryta z 16 neuronami) z funkcją Tanh w warstwie ukrytej, liniowym wyjściem i funkcją błędu MSE. Sieć uczono przez 500 epok algorytmem SGD (paczka rozmiaru 1) z krokiem $\eta = 0{,}05$ i ziarnem 42. W każdym eksperymencie zmieniano tylko jeden z tych parametrów.

5.2 Przebieg eksperymentów

Przeprowadzono następujące eksperymenty (nazwy eksperymentów pochodzą z pliku `configs.py`; każdy uruchamia się poleceniem `python main.py run <nazwa>`):

1. **Funkcja aktywacji w warstwie ukrytej** – sieć 1-16-1 z funkcją Tanh, Sigmoid lub ReLU w warstwie ukrytej, na zbiorach `cube` i `activation`, ziarno 42 (6 przebiegów; eksperyment `regression_activation`).

2. **Sieć bez warstwy ukrytej** – sieć 1-1 z liniowym wyjściem na zbiorach `cube` i `activation`, ziarno 42 (2 przebiegi).

3. **Jedna warstwa ukryta** – sieci 1-n-1 dla $n \in \{1, 2, 8, 16, 32, 64\}$, Tanh, na zbiorach `cube` i `activation`, ziarno 42 (12 przebiegów; eksperyment `regression_architecture`).

4. **Dwie warstwy ukryte** – sieci 1-2-2-1, 1-2-4-1, 1-8-8-1, 1-8-32-1, 1-32-8-1, 1-16-16-1, 1-32-32-1 i 1-64-64-1, Tanh, zbiór `cube`, ziarno 42 (8 przebiegów; eksperyment `regression_architecture`).

5. **Trzy i cztery warstwy ukryte** – sieci 1-2-2-2-1, 1-1-1-1-1-1, 1-2-2-2-2-1, 1-8-8-8-8-1 i 1-16-8-4-2-1, Tanh, zbiór `cube`, ziarno 42 (5 przebiegów; eksperyment `regression_architecture`).

6. **Funkcja błędu i aktywacja wyjścia** – sieć 1-16-1 (Tanh) w wariantach Linear + MSE, Sigmoid + MSE i Sigmoid + BCE, zbiór `cube`, ziarna 1, 2 i 3 (9 przebiegów; eksperyment `regression_loss`).

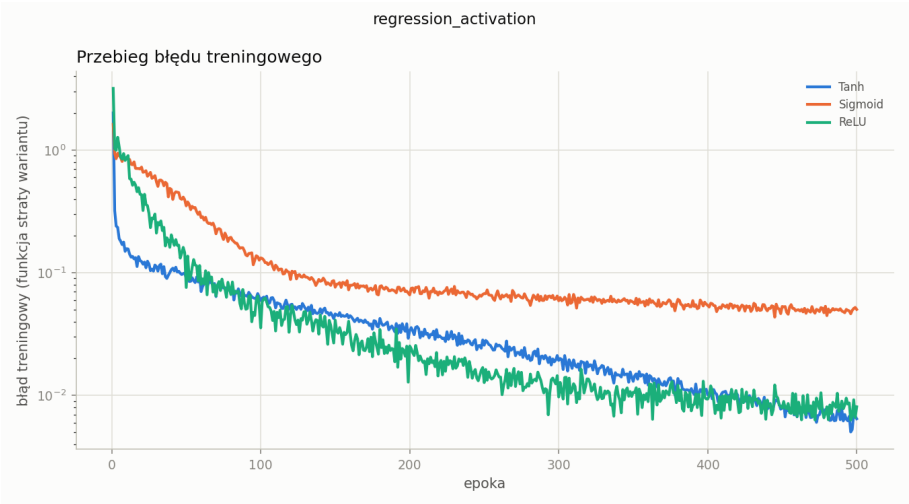
We wszystkich eksperymentach pozostałe parametry były takie jak w konfiguracji bazowej. Dla każdego przebiegu program zapisuje wykres predykcji na zakresie uczącym i na całym zbiorze testowym, przebieg błędu uczenia w kolejnych epokach, wartości MSE i MAE oraz nauczony model, a dla każdego eksperymentu dodatkowo wykresy porównujące warianty.

5.3 Wpływ funkcji aktywacji

Porównano trzy funkcje aktywacji w warstwie ukrytej: sigmoidalną, tangens hiperboliczny (Tanh) i ReLU. Na wyjściu we wszystkich przypadkach była funkcja liniowa, ponieważ w regresji wyjście nie powinno być ograniczone. Wyniki przedstawia tabela 2.

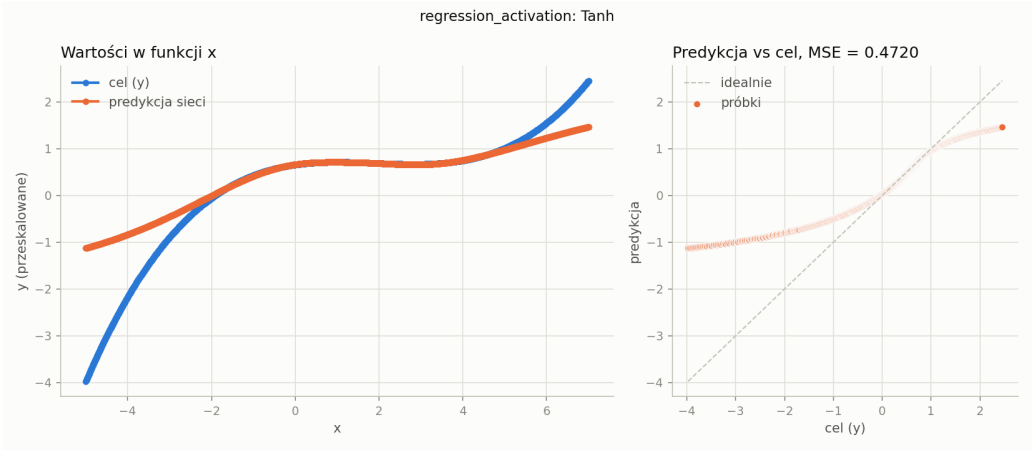
Tabela 2: Regresja, sieć 1-16-1, różne funkcje aktywacji w warstwie ukrytej (ziarno 42). MSE na zakresie uczącym podano w jednostkach 10^{-4}.

		zakres uczący		cały zbiór testowy	
Zbiór	Aktywacja	MSE [10^{-4}]	MAE	MSE	MAE
`cube`	Tanh	**0,8**	**0,0064**	**0,472**	**0,327**
	Sigmoid	7,1	0,0218	0,695	0,422
	ReLU	1,3	0,0092	0,635	0,396
`activation`	Tanh	**<0,1**	**0,0013**	**0,0007**	**0,013**
	Sigmoid	0,3	0,0047	0,0018	0,024
	ReLU	0,1	0,0018	0,0104	0,046

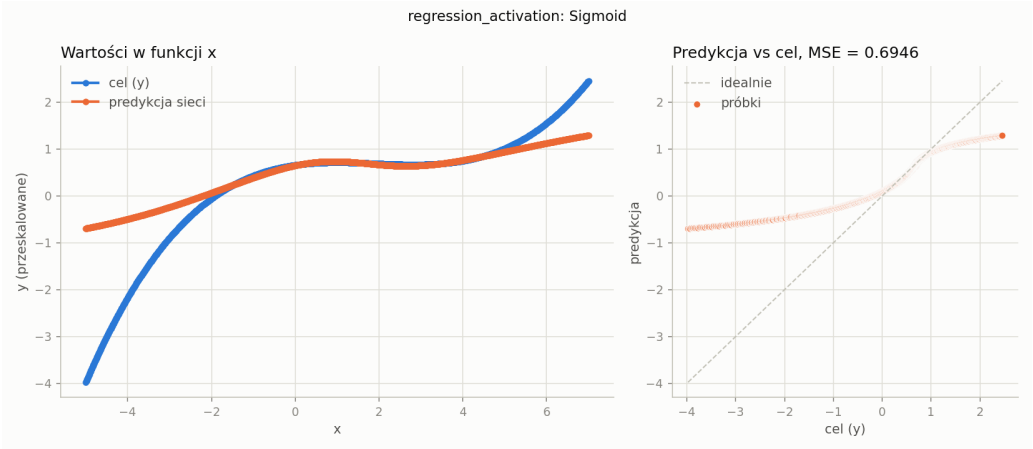


Rysunek 2: Przebieg błędu uczenia (skala logarytmiczna) dla różnych funkcji aktywacji, zbiór `cube`.

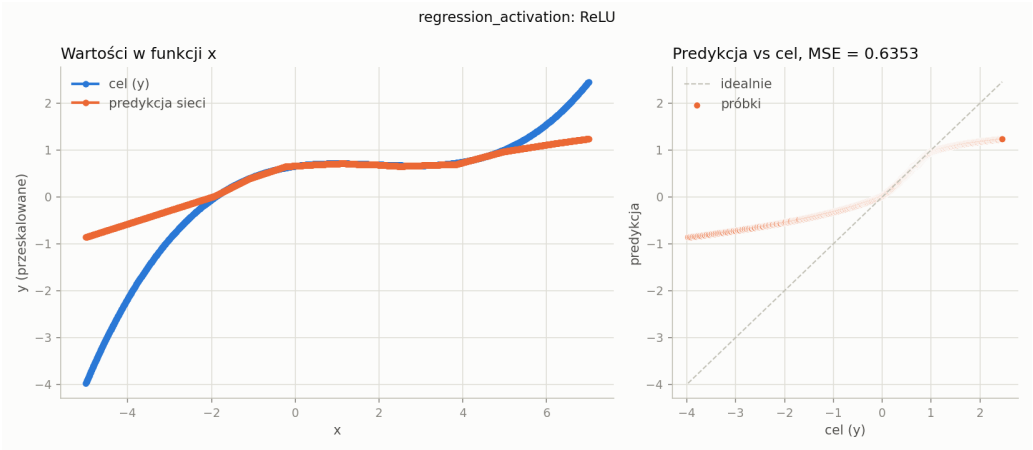
Przebieg błędu (rysunek 2) pokazuje wyraźne różnice w szybkości uczenia. Tanh obniża błąd najszybciej: już po kilku epokach spada on poniżej 0,2. ReLU początkowo uczy się wolniej, ale po ok. 100 epokach dogania Tanh i obie funkcje kończą z podobnym błędem uczenia (0,0065 i 0,0081). Funkcja sigmoidalna uczy się najwolniej i po ok. 200 epokach błąd przestaje wyraźnie maleć (końcowy błąd 0,050, czyli ok. 8 razy większy). Wynika to z małej pochodnej funkcji sigmoidalnej (maksymalnie 0,25, wobec 1 dla Tanh i ReLU), która zmniejsza gradient propagowany wstecz [4]. Dodatkowo wartości funkcji sigmoidalnej nie są wyśrodkowane wokół zera, co według [4] spowalnia uczenie.



(a) Tanh



(b) Sigmoid



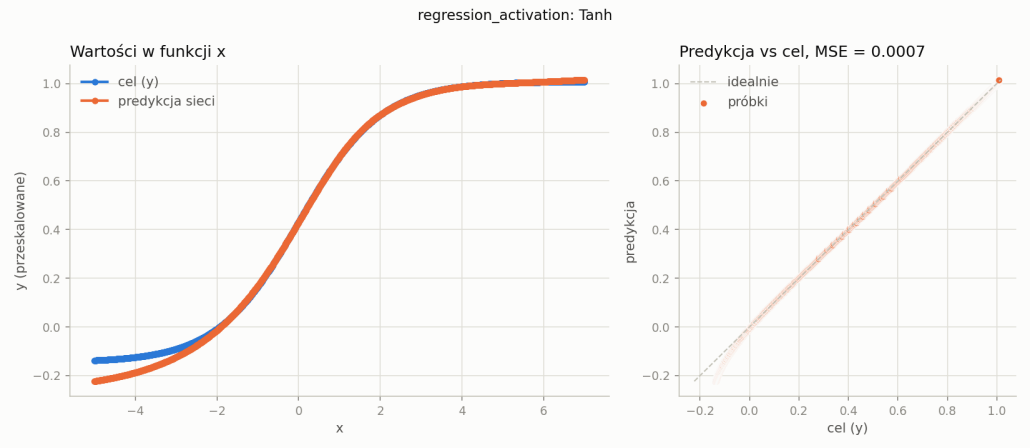
(c) ReLU

Rysunek 3: Predykcje na całym zbiorze testowym `cube` dla różnych funkcji aktywacji (niebieski – funkcja docelowa, pomarańczowy – predykcja sieci).

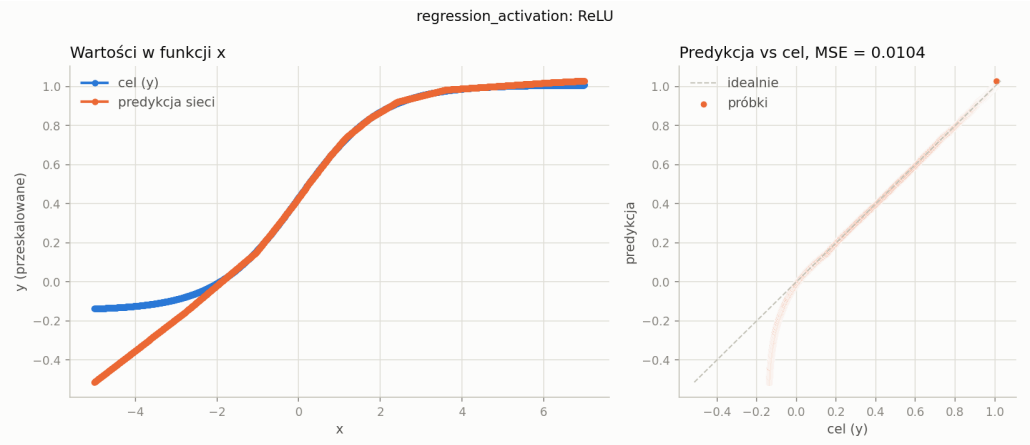
Rysunek 3 pokazuje, że wszystkie trzy sieci dobrze odtwarzają funkcję w zakresie uczącym, a różnice pojawiają się poza nim. Żadna sieć nie odtwarza sześciennego wzrostu funkcji: z Tanh i Sigmoid predykcja poza zakresem uczącym łagodnie się wypłaszcza (funkcje te nasycają się dla dużych argumentów), a z ReLU przechodzi w linię prostą (sieć z ReLU jest funkcją kawałkami liniową). Duże wartości MSE na całym zbiorze testowym (0,47–0,70) wynikają niemal wyłącznie z obszarów $x < -2$ i $x > 5$.

Funkcje aktywacji różnią się też sposobem interpolacji. Każdy neuron z ReLU wnosi do wyjścia sieci funkcję złożoną z dwóch odcinków prostych, więc cała sieć jest funkcją kawałkami liniową. Także w zakresie uczącym predykcja z ReLU składa się z odcinków prostych połączonych „załamaniami", widocznymi szczególnie w miejscach, gdzie funkcja docelowa zmienia nachylenie. Sieci z Tanh i Sigmoid są sumą gładkich krzywych, więc ich predykcja jest gładka i lepiej oddaje łagodnie zakrzywiony kształt funkcji.

Na zbiorze `activation` (rysunek 4) najlepiej działa Tanh, a sieć dopasowuje funkcję prawie idealnie także poza zakresem uczącym, ponieważ funkcja docelowa sama ma kształt funkcji aktywacji i tak samo wypłaszcza się na brzegach. Sieć z ReLU poza zakresem uczącym znów przedłuża funkcję liniowo: dla $x < -2$ predykcja maleje, podczas gdy funkcja docelowa jest stała. Stąd jej błąd na całym zbiorze testowym jest 14 razy większy niż dla Tanh.



(a) Tanh



(b) ReLU

Rysunek 4: Predykcje na całym zbiorze testowym `activation`.

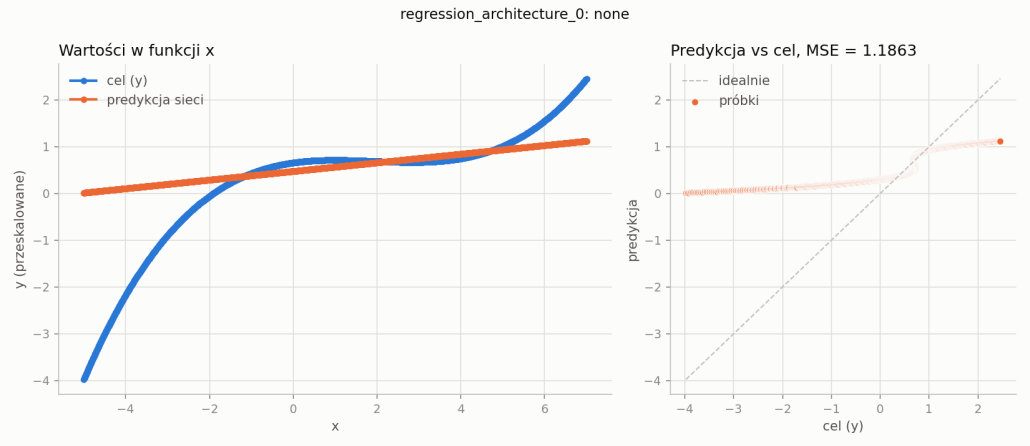
5.4 Wpływ liczby warstw ukrytych i ich liczności

Zbadano sieci z 0, 1, 2, 3 i 4 warstwami ukrytymi o różnej liczbie neuronów, przy aktywacji Tanh i pozostałych parametrach jak w konfiguracji bazowej. Wyniki dla zbioru `cube` przedstawia tabela 3, a dla zbioru `activation` tabela 4.

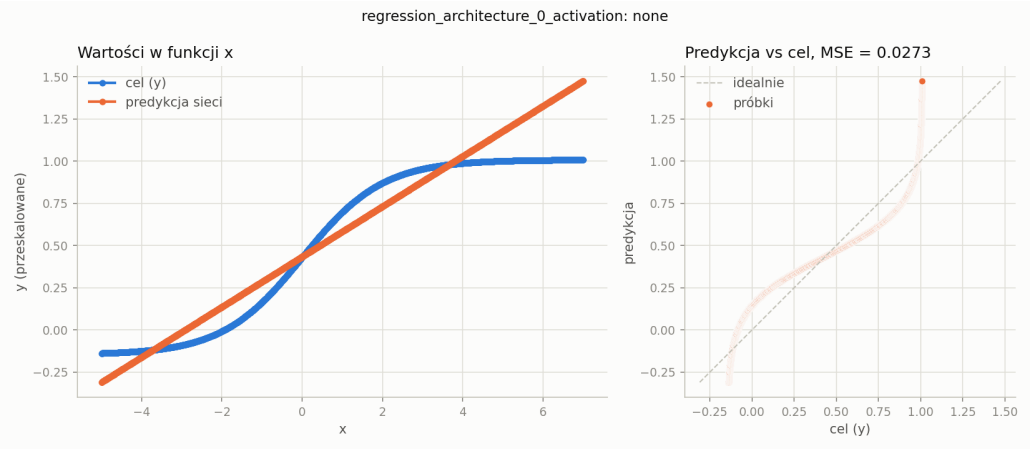
Tabela 3: Regresja na zbiorze `cube`, Tanh, różne architektury (ziarno 42). MSE na zakresie uczącym podano w jednostkach 10^{-4}. Pogrubiono najlepszy wynik w każdej kolumnie.

			zakres uczący		cały zbiór testowy	
Warstwy	Architektura	Parametry	MSE [10^{-4}]	MAE	MSE	MAE
0	1-1	2	137,3	0,1006	1,186	0,611
1	1-1-1	4	37,2	0,0413	0,724	0,463
	1-2-1	7	8,9	0,0227	0,697	0,438
	1-8-1	25	1,5	0,0099	0,536	0,352
	1-16-1	49	0,8	0,0064	0,472	0,327
	1-32-1	97	1,1	0,0079	0,432	0,313
	1-64-1	193	0,8	0,0073	**0,395**	**0,294**
2	1-2-2-1	13	2,7	0,0130	0,538	0,367
	1-2-4-1	21	10,0	0,0242	0,834	0,474
	1-8-8-1	97	1,6	0,0098	0,548	0,362
	1-8-32-1	337	0,8	0,0063	0,521	0,339
	1-32-8-1	337	0,6	0,0059	0,480	0,319
	1-16-16-1	321	3,9	0,0163	0,517	0,343
	1-32-32-1	1153	0,7	0,0070	0,487	0,324
	1-64-64-1	4353	0,5	0,0062	0,519	0,331
3	1-2-2-2-1	19	3,6	0,0160	0,834	0,432
4	1-1-1-1-1-1	10	37,7	0,0376	0,950	0,524
	1-2-2-2-2-1	25	37,7	0,0432	0,898	0,508
	1-8-8-8-8-1	241	**0,1**	**0,0027**	0,498	0,319
	1-16-8-4-2-1	217	1,3	0,0090	0,549	0,351

Brak warstwy ukrytej. Sieć bez warstwy ukrytej z liniowym wyjściem to zwykła regresja liniowa: może nauczyć się tylko prostej (rysunek 5). Na zbiorze `cube` jej błąd na zakresie uczącym jest ok. 170 razy większy niż dla sieci 1-16-1. Na zbiorze `activation` prosta przybliża „schodek” tylko w przybliżeniu (MSE = 0,011 na zakresie uczącym, wobec MSE rzędu 10^{-6} dla sieci z warstwą ukrytą). Do odtworzenia funkcji nieliniowej potrzebna jest co najmniej jedna warstwa ukryta z nieliniową aktywacją.



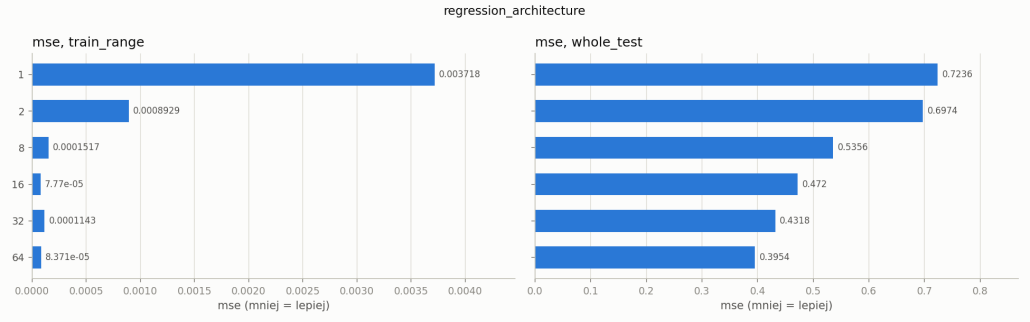
(a) Zbiór `cube`



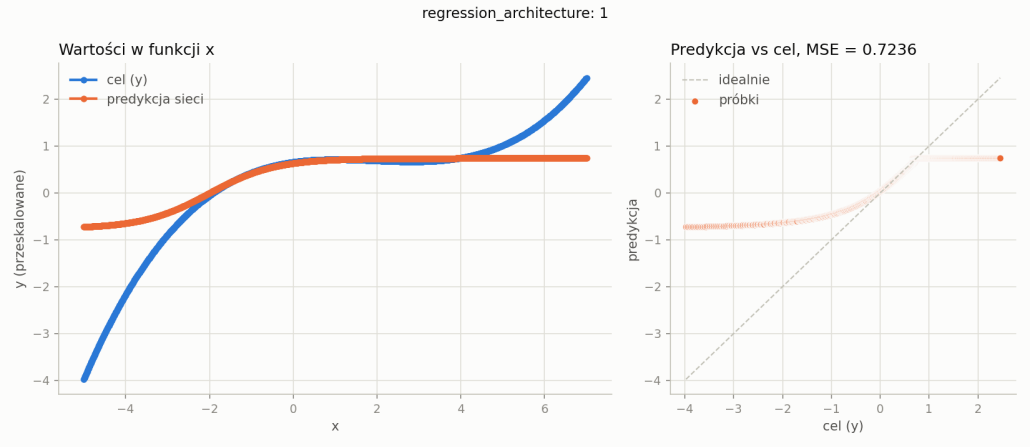
(b) Zbiór `activation`

Rysunek 5: Sieć bez warstwy ukrytej (1-1) – predykcje na całym zbiorze testowym.

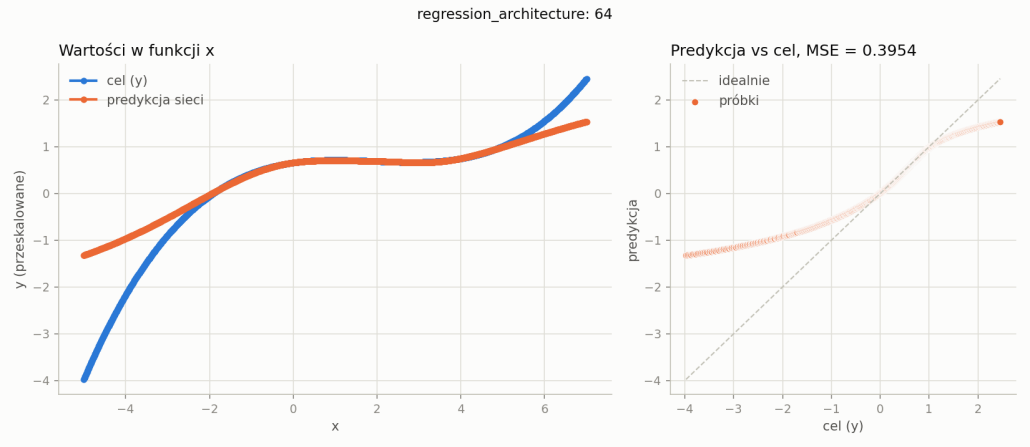
Jedna warstwa ukryta. Wraz ze wzrostem liczby neuronów błąd na zakresie uczącym szybko maleje: od $37 \cdot 10^{-4}$ dla jednego neuronu do ok. $1 \cdot 10^{-4}$ dla 16 i więcej neuronów, po czym się stabilizuje (rysunek 6). Sieć z jednym neuronem ukrytym może przedstawić tylko jedną przeskalowaną funkcję Tanh, więc nie jest w stanie odtworzyć lokalnego minimum funkcji sześciennej, a poza zakresem uczącym jest stała (rysunek 7). Błąd na całym zbiorze testowym maleje monotonicznie z liczbą neuronów (od 0,724 do 0,395), ponieważ szersza sieć dłużej „podąża” za trendem funkcji po wyjściu poza dane uczące. Sieć 1-64-1 osiągnęła najmniejszy błąd na całym zbiorze testowym spośród wszystkich badanych architektur. Zgodnie z twierdzeniem o uniwersalnej aproksymacji [7] jedna warstwa ukryta z wystarczającą liczbą neuronów wystarcza do dowolnie dokładnego przybliżenia funkcji ciągłej na ograniczonym przedziale.



Rysunek 6: MSE na zakresie uczącym (lewo) i na całym zbiorze testowym (prawo) dla sieci z jedną warstwą ukrytą, zbiór `cube`.



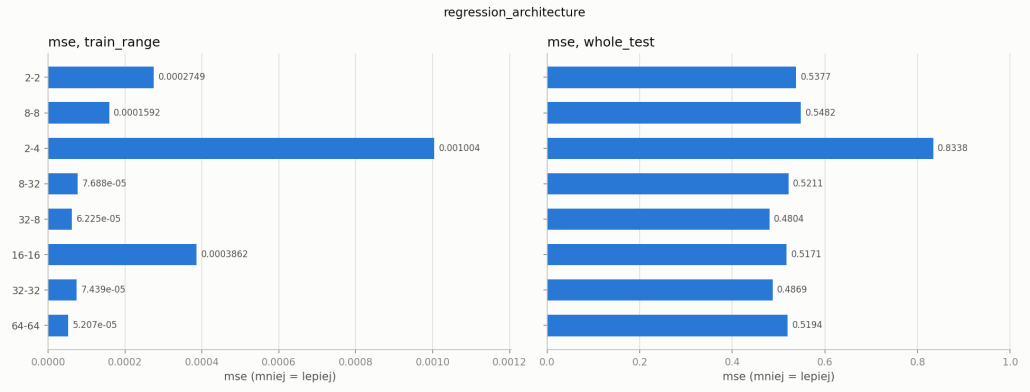
(a) 1-1-1



(b) 1-64-1

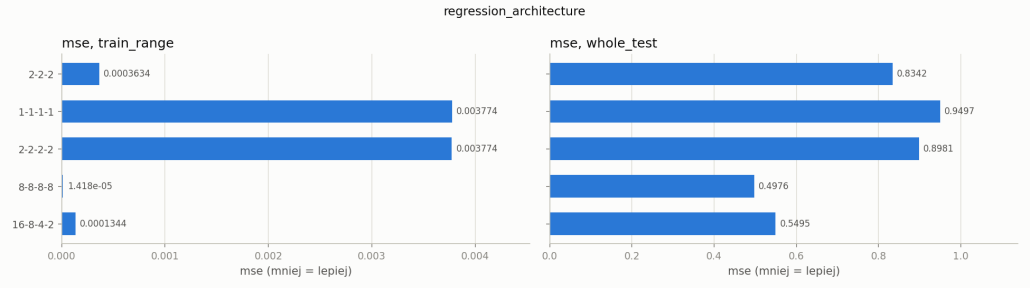
Rysunek 7: Sieć z jednym i z 64 neuronami w warstwie ukrytej, zbiór `cube`.

Dwie warstwy ukryte. Druga warstwa ukryta nie poprawiła wyników (rysunek 8). Najlepsze sieci dwuwarstwowe (1-32-8-1, 1-64-64-1) osiągają na zakresie uczącym błąd porównywalny z siecią 1-16-1 (ok. 0,5–0,8 $\cdot 10^{-4}$), mimo że mają od 7 do 90 razy więcej parametrów. Na całym zbiorze testowym żadna nie jest lepsza od sieci 1-64-1. Najgorsza okazała się sieć 1-2-4-1 z wąskim (dwuneuronowym) gardłem w pierwszej warstwie.

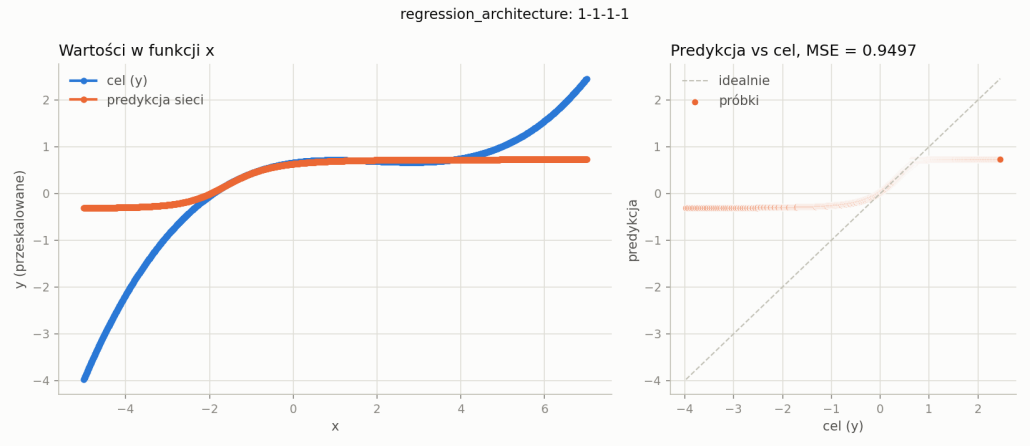


Rysunek 8: MSE dla sieci z dwiema warstwami ukrytymi, zbiór `cube`.

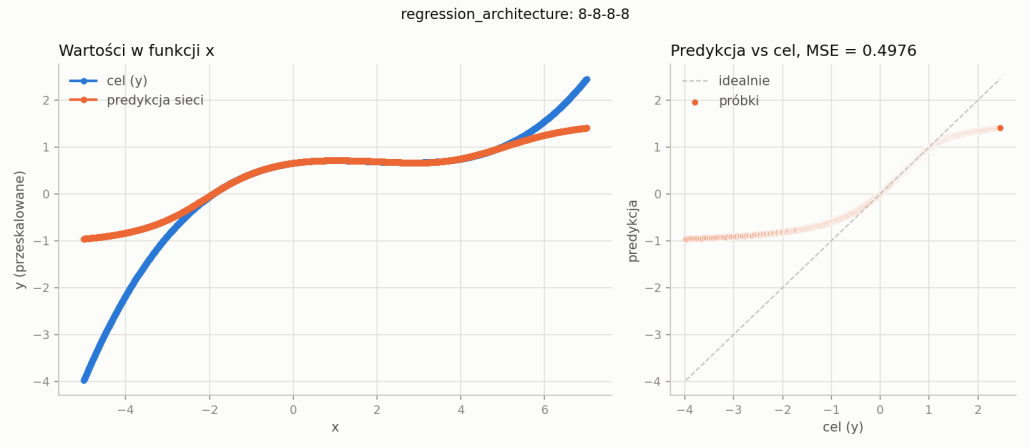
Trzy i cztery warstwy ukryte. W głębszych sieciach decydujące znaczenie ma szerokość warstw. Sieci z bardzo wąskimi warstwami (1-1-1-1-1-1 i 1-2-2-2-2-1) uczą się tak samo źle jak sieć z jednym neuronem ukrytym (MSE $\approx 37 \cdot 10^{-4}$, rysunek 10): informacja musi przejść przez kolejne jedno- lub dwuneuronowe „wąskie gardła", a gradient przy propagacji wstecznej jest wielokrotnie mnożony przez pochodne funkcji Tanh (mniejsze od 1), co osłabia uczenie początkowych warstw [2]. Sieć 1-8-8-8-8-1 osiągnęła najmniejszy błąd na zakresie uczącym spośród wszystkich badanych architektur ($0{,}1 \cdot 10^{-4}$), ale na całym zbiorze testowym nie przewyższa sieci z jedną warstwą ukrytą.



Rysunek 9: MSE dla sieci z trzema i czterema warstwami ukrytymi, zbiór `cube`.



(a) 1-1-1-1-1-1



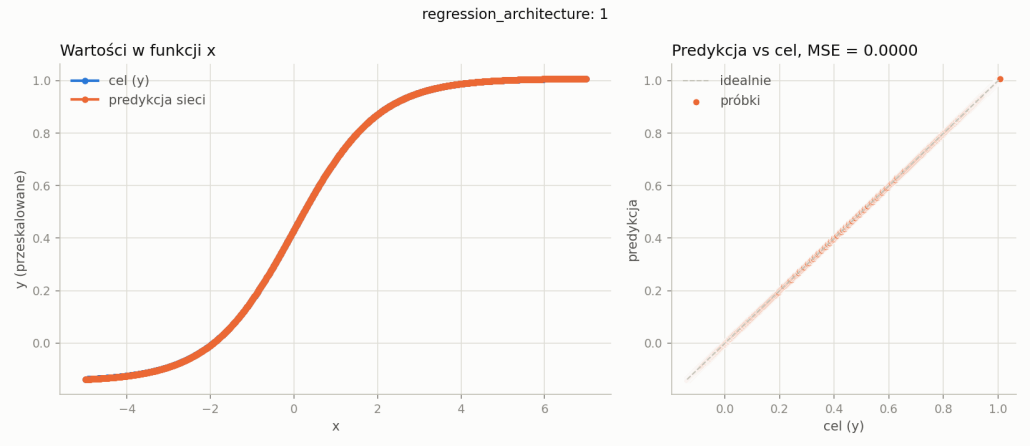
(b) 1-8-8-8-8-1

Rysunek 10: Sieci z czterema warstwami ukrytymi: wąska i szeroka, zbiór `cube`.

Zbiór `activation`. Na tym zbiorze najlepszy wynik osiągnęła najmniejsza sieć, 1-1-1 (tabela 4, rysunek 11): błąd jest praktycznie zerowy także poza zakresem uczącym. Funkcja docelowa ma kształt funkcji aktywacji, więc jeden neuron z Tanh może ją odtworzyć dokładnie. Większe sieci mają więcej parametrów do dopasowania przy tej samej liczbie epok i osiągają nieco większe błędy, szczególnie poza zakresem uczącym.

Tabela 4: Regresja na zbiorze `activation`, Tanh, różna liczba neuronów (ziarno 42).

		zakres uczący	cały zbiór testowy	
Architektura	Parametry	MAE	MSE	MAE
1-1 (bez warstwy ukrytej)	2	0,0923	0,02726	0,1299
1-1-1	4	**0,0001**	**0,00000**	**0,0002**
1-2-1	7	0,0056	0,00354	0,0356
1-8-1	25	0,0016	0,00037	0,0122
1-16-1	49	0,0013	0,00072	0,0131
1-32-1	97	0,0025	0,00144	0,0215
1-64-1	193	0,0050	0,00277	0,0267



Rysunek 11: Sieć 1-1-1 na zbiorze `activation` – predykcja pokrywa się z funkcją docelową na całym zbiorze testowym.

5.5 Wpływ funkcji błędu

Porównano dwie funkcje błędu: błąd średniokwadratowy (MSE) i binarną entropię krzyżową (BCE). BCE jest określona tylko dla wyjść z przedziału $(0, 1)$, dlatego wymaga funkcji sigmoidalnej na wyjściu. Porównano trzy warianty:

- **Linear + MSE** – standardowe połączenie dla regresji (wariant bazowy),
- **Sigmoid + MSE** – to samo co wyżej, ale z wyjściem ograniczonym do $(0, 1)$,
- **Sigmoid + BCE** – to samo wyjście co wyżej, ale inna funkcja błędu.

Porównanie wariantów 2 i 3 pokazuje wpływ samej funkcji błędu, a porównanie wariantów 1 i 2 wpływ funkcji aktywacji na wyjściu. Każdy wariant uczono trzy razy z ziarnami 1, 2 i 3. Tabela 5 podaje średnią i odchylenie standardowe.

Tabela 5: Regresja na zbiorze `cube`, sieć 1-16-1 (Tanh), różne wyjścia i funkcje błędu. Średnia ± odchylenie standardowe z 3 ziaren. MSE na zakresie uczącym w jednostkach 10^{-4}.

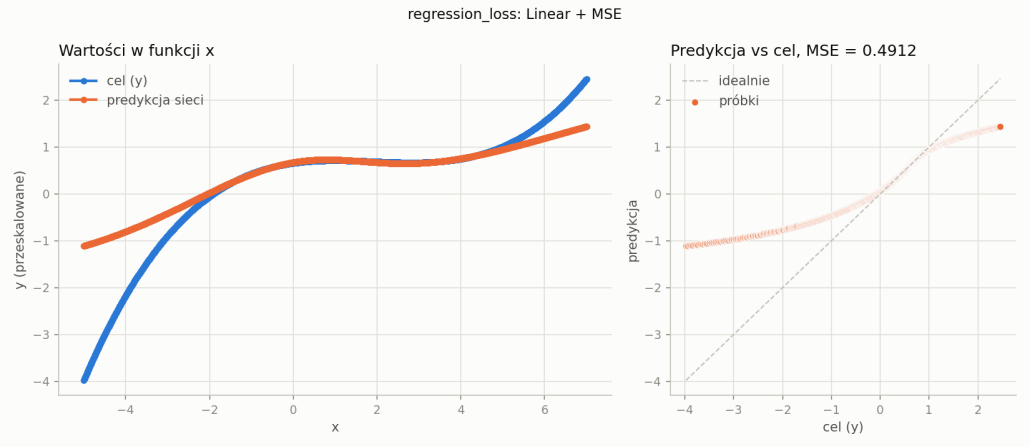
	zakres uczący		cały zbiór testowy	
Wariant	MSE $[10^{-4}]$	MAE	MSE	MAE
Linear + MSE	$\mathbf{1{,}8 \pm 0{,}7}$	$\mathbf{0{,}010 \pm 0{,}002}$	$\mathbf{0{,}498 \pm 0{,}017}$	$\mathbf{0{,}340 \pm 0{,}006}$
Sigmoid + MSE	$12{,}1 \pm 0{,}9$	$0{,}025 \pm 0{,}002$	$1{,}137 \pm 0{,}002$	$0{,}557 \pm 0{,}001$
Sigmoid + BCE	$3{,}4 \pm 0{,}3$	$0{,}014 \pm 0{,}001$	$1{,}106 \pm 0{,}000$	$0{,}532 \pm 0{,}001$



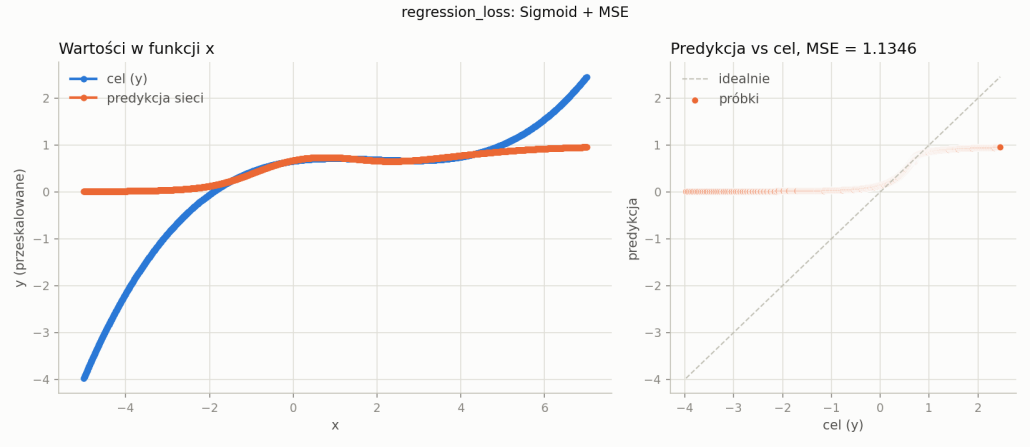
Rysunek 12: MSE dla różnych wyjść i funkcji błędu (słupki błędów – odchylenie standardowe z 3 ziaren).

MSE a BCE. Przy tym samym wyjściu sigmoidalnym funkcja BCE dała 3,5 razy mniejszy błąd na zakresie uczącym niż MSE. Wynika to z postaci gradientu: dla MSE pochodna błędu względem pobudzenia neuronu wyjściowego zawiera czynnik $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, który jest bliski zera, gdy wyjście zbliża się do 0 lub 1. Dla BCE czynnik ten się skraca i gradient wynosi po prostu $\hat{y} - y$ [3], więc sieć uczy się szybciej także dla skrajnych wartości. Wartości błędu uczenia dla obu funkcji nie są ze sobą porównywalne (dla BCE końcowy błąd uczenia wynosi 57,6, a dla MSE 0,068), ponieważ dla celów z wnętrza przedziału $(0, 1)$ minimum BCE nie jest równe zeru. Dlatego warianty porównano za pomocą tej samej miary (MSE, MAE) na zbiorze testowym.

Wyjście liniowe a sigmoidalne. Najlepsze wyniki daje jednak wyjście liniowe z MSE. Wyjście sigmoidalne ma dwie wady. Pierwsza to wolniejsze uczenie przy MSE, opisane wyżej. Druga, ważniejsza, to ograniczenie wyjścia do przedziału $(0, 1)$: sieć nie może zwrócić wartości spoza zakresu, który widziała w zbiorze uczącym. Na rysunku 13 widać, że poza zakresem uczącym predykcja z wyjściem sigmoidalnym jest „obcięta" do wartości 0 i 1, podczas gdy przeskalowane wartości testowe sięgają od -4 do 2,4. Dlatego błąd na całym zbiorze testowym jest ponad dwukrotnie większy niż dla wyjścia liniowego. W regresji należy więc stosować liniową funkcję aktywacji na wyjściu.



(a) Linear + MSE



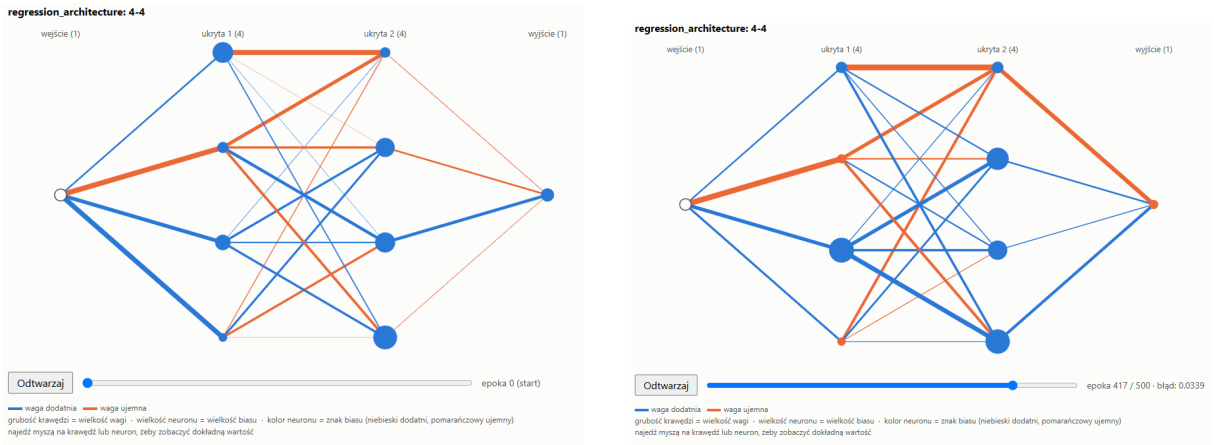
(b) Sigmoid + MSE

Rysunek 13: Predykcje na całym zbiorze testowym `cube` dla wyjścia liniowego i sigmoidalnego (jeden z trzech przebiegów).

5.6 Ewolucja wag

Program pozwala obejrzeć, jak zmieniają się wagi sieci w trakcie uczenia. Dla wybranego przebiegu zapisywany jest interaktywny plik `network.html`, w którym suwak przesuwa się po kolejnych epokach. Grubość i kolor krawędzi odpowiadają wartości wagi (kolor niebieski – waga dodatnia, pomarańczowy – ujemna), a wielkość neuronu wartości biasu. Skala jest stała dla wszystkich epok, więc ten sam wygląd krawędzi oznacza tę samą wartość wagi w każdej klatce.

Ewolucję wag pokazano na przykładzie sieci 1-4-4-1 (dwie warstwy ukryte po 4 neurony, Tanh, 33 parametry) uczonej na zbiorze `cube_100` w konfiguracji bazowej (500 epok, $\eta = 0{,}05$, ziarno 42). Sieć osiągnęła MSE = 0,0004 na zakresie uczącym i 0,545 na całym zbiorze testowym, czyli wyniki zbliżone do innych sieci z dwiema warstwami ukrytymi (tabela 3). Rysunek 14 przedstawia wagi przed rozpoczęciem uczenia i po 500 epokach.



(a) Wagi i biasy początkowe (przed uczeniem)

(b) Wagi i biasy po 500 epokach

Rysunek 14: Wagi sieci 1-4-4-1 (Tanh) uczonej na zbiorze `cube_100`. Grubość i kolor krawędzi odpowiadają wartości wagi, wielkość neuronu wartości biasu.

Najważniejsze obserwacje:

- **Wagi zmieniają się najszybciej na początku uczenia.** Suma bezwzględnych zmian wszystkich 33 parametrów wynosi 2,26 w pierwszej epoce, 0,58 w drugiej i 0,22 w trzeciej, a w setnej już tylko ok. 0,1 i w czterechsetnej 0,025. Połowę całkowitej zmiany wag sieć wykonuje w pierwszych 30 epokach, a 90% w pierwszych 170. Odpowiada to przebiegowi błędu uczenia, który najszybciej spada w pierwszych epokach (MSE na zbiorze uczącym 0,023 po pierwszej i 0,012 po drugiej epoce).

- **Mimo coraz mniejszych zmian wag błąd dalej maleje**, od 0,0032 po 100 epokach do 0,0005 po 500. W drugiej części uczenia sieć wykonuje już tylko drobne korekty, które dopracowują kształt dopasowanej funkcji.

- **Najbardziej zmieniają się wagi warstwy wyjściowej.** Waga pierwszego neuronu drugiej warstwy ukrytej zmienia się z $-0{,}09$ na $-0{,}90$, a dwie wagi wyjściowe zmieniają znak ($-0{,}34 \to 0{,}34$ i $-0{,}08 \to 0{,}57$). Wagi warstwy wejściowej zmieniają się w mniejszym stopniu (największa zmiana: $0{,}94 \to 0{,}41$), a duże zmiany zachodzą też w biasach pierwszej warstwy ukrytej (np. $0{,}47 \to 1{,}12$ i $0{,}78 \to 0{,}19$). Biasy pierwszej warstwy decydują o tym, w którym miejscu osi x dany neuron „przełącza się", więc ich zmiana przesuwa zakrzywienia funkcji sieci w miejsca, w których zakrzywia się funkcja docelowa.

- **Wartości wag pozostają ograniczone.** Największa wartość bezwzględna wagi wynosi 1,04 na początku i 1,21 na końcu uczenia. Wagi nie rosną nadmiernie, a neurony z Tanh

nie wchodzą w obszar nasycenia, w którym uczenie by się zatrzymało. Łącznie 4 z 24 wag zmieniły znak.

5.7 Powtarzalność wyników

Wariant Linear + MSE z tabeli 5 ma dokładnie tę samą konfigurację co sieć bazowa z tabel 2 i 3 (1-16-1, Tanh), różni się tylko ziarnem. Pozwala to oszacować, jak bardzo wynik zależy od losowej inicjalizacji wag:

- ziarno 42: MSE na zakresie uczącym $0{,}8 \cdot 10^{-4}$, na całym zbiorze testowym 0,472,
- ziarna 1, 2, 3: MSE na zakresie uczącym $(1{,}8 \pm 0{,}7) \cdot 10^{-4}$, na całym zbiorze testowym $0{,}498 \pm 0{,}017$.

Błąd na całym zbiorze testowym jest stabilny (różnice rzędu 5%), natomiast bardzo mały błąd na zakresie uczącym zmienia się między ziarnami nawet dwukrotnie. Oznacza to, że różnice między architekturami w tabeli 3 rzędu $(0{,}5\text{–}2) \cdot 10^{-4}$ (np. 1-8-32-1 i 1-32-32-1) mieszczą się w zmienności losowej i nie należy ich interpretować jako przewagi jednej architektury nad drugą. Wiarygodne są natomiast różnice o rząd wielkości lub większe, np. między siecią bez warstwy ukrytej, siecią z jednym neuronem i siecią z 16 neuronami albo między wąskimi i szerokimi sieciami głębokimi. Wyniki eksperymentów z funkcją błędu (3 ziarna) mają małe odchylenia standardowe, więc ich wnioski są pewne.

5.8 Podsumowanie wyników regresji

- Wszystkie sieci z co najmniej jedną warstwą ukrytą o wystarczającej szerokości bardzo dobrze odtwarzają funkcję w zakresie danych uczących. Żadna nie potrafi jednak poprawnie ekstrapolować funkcji sześciennej: zachowanie poza zakresem uczącym wynika z kształtu funkcji aktywacji (wypłaszczenie dla Tanh i Sigmoid, linia prosta dla ReLU), a nie z danych.
- Spośród funkcji aktywacji najlepiej sprawdził się Tanh (najszybsze uczenie, najmniejsze błędy). ReLU osiąga podobny błąd uczenia, ale gorzej ekstrapoluje. Sigmoid uczy się wyraźnie najwolniej.
- Sieć bez warstwy ukrytej jest regresją liniową i nie nadaje się do funkcji nieliniowych. Jedna warstwa ukryta z 16–64 neuronami wystarcza. Kolejne warstwy nie poprawiły wyników, a sieci głębokie z bardzo wąskimi warstwami (1–2 neurony) uczą się źle.
- Na wyjściu sieci regresyjnej powinna być funkcja liniowa. Przy wyjściu sigmoidalnym funkcja BCE uczy się szybciej niż MSE, ale wyjście ograniczone do $(0, 1)$ uniemożliwia przewidywanie wartości spoza zakresu zbioru uczącego.
- Małe różnice między konfiguracjami (rzędu 10^{-4} MSE) są porównywalne ze zmiennością wynikającą z ziarna generatora, więc do ich rozstrzygnięcia potrzebne są powtórzenia z wieloma ziarnami.

6 Wyniki – klasyfikacja

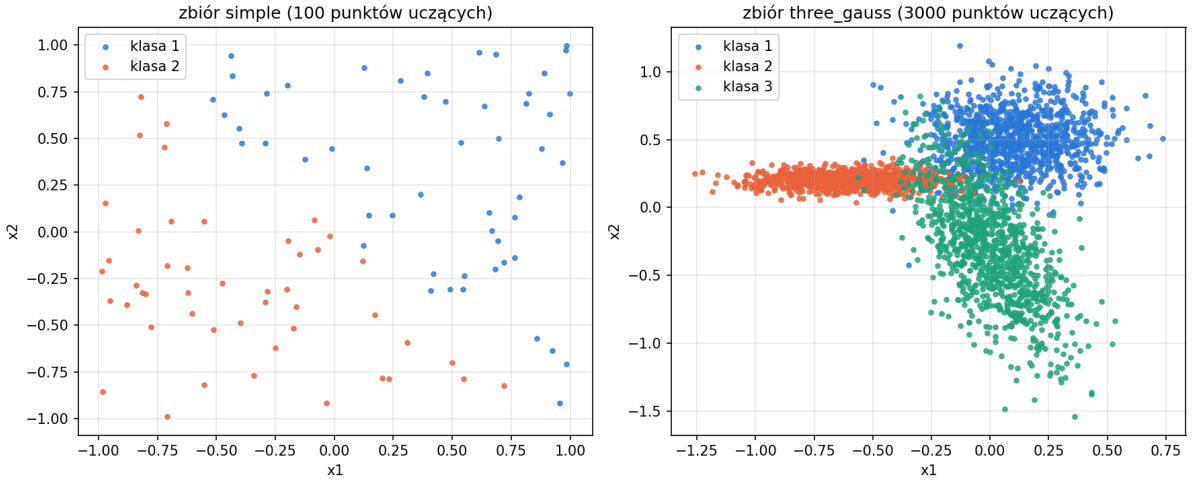
6.1 Zbiory danych i sposób oceny

Zadanie klasyfikacji polega na przypisaniu punktowi z płaszczyzny $\mathbb{R}^2$ jednej z kilku klas. Wykorzystano dwa zbiory (rysunek 15):

- `simple` – dwie klasy, punkty rozłożone równomiernie w kwadracie $[-1, 1]^2$ i rozdzielone prostą (w przybliżeniu $x_1 + x_2 = 0$). Zbiór jest liniowo separowalny, czyli klasy da się bezbłędnie rozdzielić jedną prostą.

- `three_gauss` – trzy klasy, każda wylosowana z innego rozkładu normalnego. Klasa 2 tworzy wydłużony, poziomy pas, a klasy 1 i 3 skupiska nad nim i pod nim. Wszystkie trzy rozkłady nachodzą na siebie w okolicy punktu (0; 0,2), więc część punktów leży „w obszarze innej klasy". Takich punktów nie da się poprawnie sklasyfikować żadną granicą decyzyjną.

Każdy zbiór jest dostępny w kilku rozmiarach. W zbiorze `simple` liczba w nazwie oznacza liczbę wszystkich punktów (np. `simple_1000` ma 1000 punktów uczących), a w zbiorze `three_gauss` liczbę punktów w każdej klasie (np. `three_gauss_1000` ma 3000 punktów uczących). Zbiór testowy ma zawsze ten sam rozmiar i rozkład co zbiór uczący.



Rysunek 15: Zbiory uczące do klasyfikacji: `simple_100` (lewo) i `three_gauss_1000` (prawo).

Przygotowanie danych i miara jakości. Współrzędne punktów są standaryzowane (średnia 0, odchylenie standardowe 1) na podstawie statystyk zbioru uczącego, a klasy zakodowane w postaci *one-hot*. Sieć ma tyle wyjść, ile klas. Domyślnie na wyjściu stosowano funkcję Softmax, a jako funkcję błędu entropię krzyżową (CCE). Klasą przewidzianą przez sieć jest wyjście o największej wartości. Jakość mierzono dokładnością (*accuracy*), czyli odsetkiem poprawnie sklasyfikowanych punktów zbioru testowego. Błąd uczenia na wykresach to suma wartości funkcji błędu po wszystkich punktach zbioru uczącego.

Konfiguracja bazowa. Jeśli nie zaznaczono inaczej, używano sieci z dwiema warstwami ukrytymi po 8 neuronów (2-8-8-3 dla `three_gauss`) z funkcją sigmoidalną w warstwach ukrytych, wyjściem Softmax i funkcją błędu CCE. Sieć uczono przez 20 epok algorytmem SGD (paczka rozmiaru 1) z krokiem $\eta = 0{,}1$ i ziarnem 42.

6.2 Przebieg eksperymentów

Przeprowadzono dwie serie eksperymentów. Seria główna (20 epok, zbiory `simple` i `three_gauss_1000`):

1. **Sieć bez warstwy ukrytej** – sieć 2-K (K – liczba klas) na zbiorach `simple_100` i `simple_10000` (dla tego drugiego krok 0,1 i 0,01) oraz `three_gauss_100` i `three_gauss_10000`, ziarno 42.

2. **Jedna warstwa ukryta** – sieci 2-n-K dla $n \in \{1, 2, 4, 8, 16\}$ na zbiorach `simple_100`, `simple_1000` i `three_gauss_1000`, ziarno 42.

3. **Dwie, trzy i cztery warstwy ukryte** – sieci 2-2-2-3, 2-8-8-3, 2-8-4-3, 2-4-4-4-3, 2-4-4-4-4-3 i 2-32-32-32-32-3 na zbiorze `three_gauss_1000`, ziarno 42.

4. **Funkcja aktywacji w warstwach ukrytych** – sieć 2-8-8-3 z funkcją Tanh, Sigmoid lub ReLU na zbiorze `three_gauss_1000`, z krokiem 0,1 i 0,01, ziarno 42.

5. **Funkcja błędu** – sieć 2-8-8-3 z wyjściem Softmax i funkcją błędu CCE lub BCE na zbiorze `three_gauss_1000`, ziarna 1, 2 i 3.

Seria dodatkowa, z większą liczbą epok i powtórzeniami (zbiór `three_gauss_100`, 300 epok, sieć bazowa 2-8-3 z funkcją sigmoidalną, ziarna 1, 2 i 3):

6. **Funkcja aktywacji** – Tanh, Sigmoid i ReLU w warstwie ukrytej.

7. **Liczba neuronów** – sieci 2-n-3 dla $n \in \{1, 2, 8, 16, 32, 64\}$.

8. **Liczba warstw** – sieci 2-3 (bez warstwy ukrytej), 2-8-8-3, 2-8-8-8-3 i 2-8-8-8-8-3.

9. **Wyjście i funkcja błędu** – Softmax + CCE, Softmax + MSE, Sigmoid + MSE i Sigmoid + BCE.

Sprawdzono też wpływ liczby epok na zbiorze `simple_100` (sieci 2-n-2 dla $n \in \{1, 2, 8, 16, 32, 64\}$, uczone przez 20, 80 i 300 epok). Dla każdego przebiegu program zapisuje wykres obszarów decyzyjnych na tle punktów testowych, przebieg błędu uczenia i dokładności w kolejnych epokach oraz nauczony model.

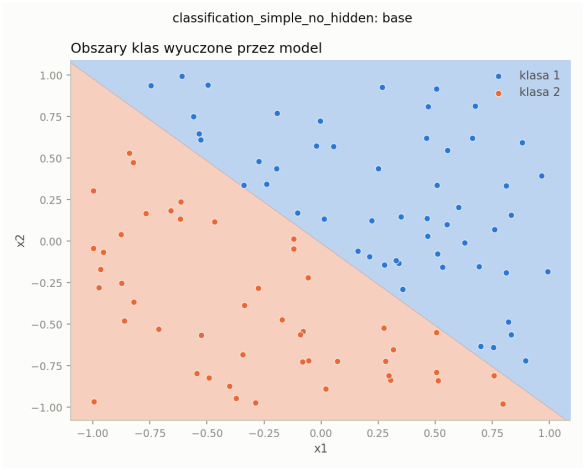
6.3 Sieć bez warstwy ukrytej

Sieć bez warstwy ukrytej z wyjściem Softmax to regresja logistyczna (wielomianowa): każde wyjście jest liniową funkcją wejść przepuszczoną przez Softmax. Granice między obszarami klas są więc zawsze odcinkami prostych. Wyniki przedstawia tabela 6.

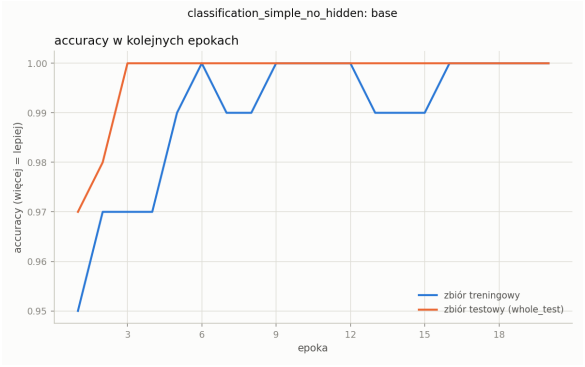
Tabela 6: Sieć bez warstwy ukrytej (2-K), Softmax + CCE, ziarno 42.

Zbiór	Pkt. uczące	Pkt. testowe	η	Epoki	Dokładność [%]
`simple_100`	100	100	0,1	20	100,00
`simple_10000`	10 000	10 000	0,1	20	99,62
`simple_10000`	10 000	10 000	0,01	20	99,62
`three_gauss_100`	300	300	0,1	20	92,33
`three_gauss_100`	300	300	0,1	300	93,00
`three_gauss_10000`	30 000	30 000	0,1	20	92,31

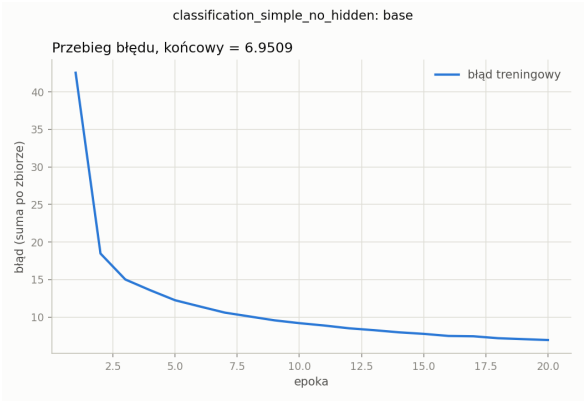
Zbiór `simple` jest liniowo separowalny, dlatego już sieć bez warstwy ukrytej klasyfikuje go bezbłędnie (rysunek 16). Na dużym zbiorze `simple_10000` pojawia się 0,38% błędów (38 z 10 000 punktów). Przy 20 epokach granica nie jest ustawiona idealnie, a przy dużej liczbie punktów część z nich leży bardzo blisko prawdziwej granicy klas. Już po trzech epokach dokładność testowa osiąga 100%, a błąd uczenia w kolejnych epokach nadal maleje. Sieć coraz pewniej przypisuje punkty do klas, chociaż granica decyzyjna już się nie zmienia.



(a) Obszary decyzyjne i punkty testowe



(b) Dokładność w kolejnych epokach



(c) Błąd uczenia w kolejnych epokach

Rysunek 16: Sieć bez warstwy ukrytej na zbiorze `simple_100`.

Na zbiorze `three_gauss` sieć bez warstwy ukrytej dzieli płaszczyznę na trzy obszary ograniczone prostymi spotykającymi się w jednym punkcie (rysunek 17). Osiąga przy tym ok. 92–93% dokładności, niezależnie od rozmiaru zbioru. Błędne klasyfikacje to prawie wyłącznie punkty z obszaru, w którym klasy na siebie nachodzą.



Rysunek 17: Sieć bez warstwy ukrytej na zbiorze `three_gauss_100` – obszary decyzyjne.

6.4 Wpływ liczby warstw ukrytych i ich liczności

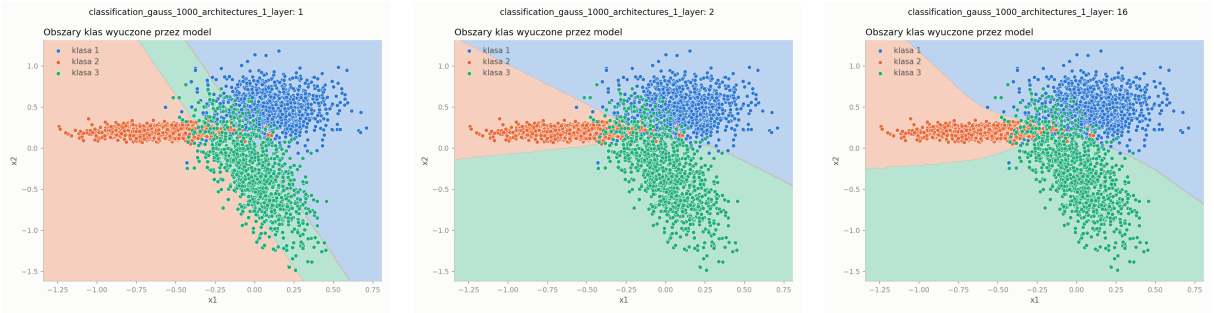
Jedna warstwa ukryta. Wyniki dla sieci z jedną warstwą ukrytą przedstawia tabela 7.

Tabela 7: Sieci z jedną warstwą ukrytą (2-n-K), Sigmoid, 20 epok, ziarno 42. Dokładność na zbiorze testowym [%].

Neurony ukryte n	`simple_100`	`simple_1000`	`three_gauss_1000`
1	98,0	**99,6**	83,73
2	**100,0**	99,2	**93,13**
4	**100,0**	99,2	92,63
8	99,0	99,2	92,37
16	99,0	99,0	93,03

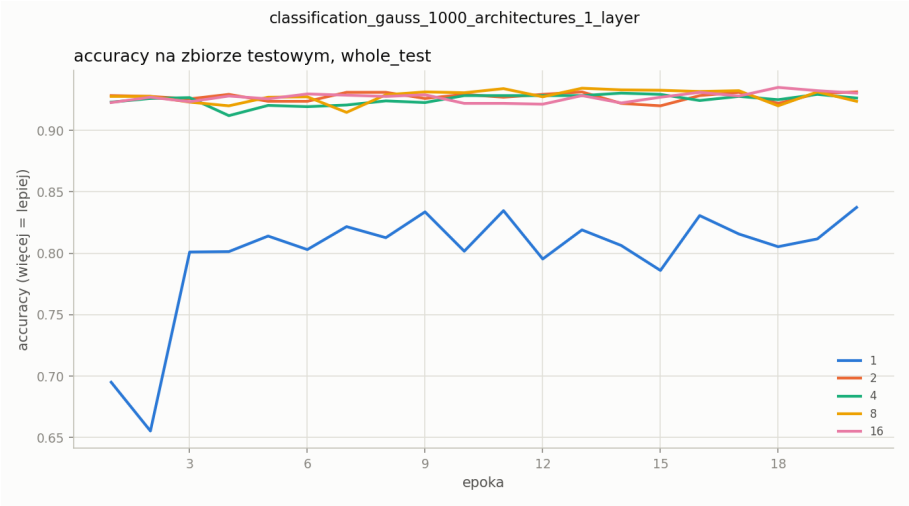
Na zbiorze `simple` wszystkie sieci osiągają 98–100%. Do rozdzielenia dwóch klas prostą wystarcza jeden neuron ukryty, a różnice między wariantami to pojedyncze punkty (na zbiorze 100-punktowym jeden punkt to 1 punkt procentowy). Podobnie liczba epok nie miała znaczenia: przy 20, 80 i 300 epokach sieci 2-n-2 dla n od 1 do 64 osiągały na `simple_100` od 98 do 100%.

Na zbiorze `three_gauss` sieć z jednym neuronem ukrytym osiąga tylko 83,7%. Wyjście takiej sieci zależy od wejścia wyłącznie przez jedną kombinację liniową $w_1x_1 + w_2x_2 + b$, więc granice decyzyjne muszą być do siebie równoległe, a klasy ułożone w „pasy” jedna za drugą (rysunek 18a). Trzech klas położonych tak jak w `three_gauss` nie da się w ten sposób rozdzielić. Już dwa neurony ukryte pozwalają uzyskać granice nierównoległe i dokładność 93,1%. Dalsze zwiększanie liczby neuronów nie poprawia wyniku: pozostałe ok. 7% błędów to punkty z obszaru, w którym klasy na siebie nachodzą.



(a) 2-1-3 (83,7%) (b) 2-2-3 (93,1%) (c) 2-16-3 (93,0%)

Rysunek 18: Obszary decyzyjne sieci z jedną warstwą ukrytą na zbiorze `three_gauss_1000`.



Rysunek 19: Dokładność testowa w kolejnych epokach dla sieci z jedną warstwą ukrytą (`three_gauss_1000`).

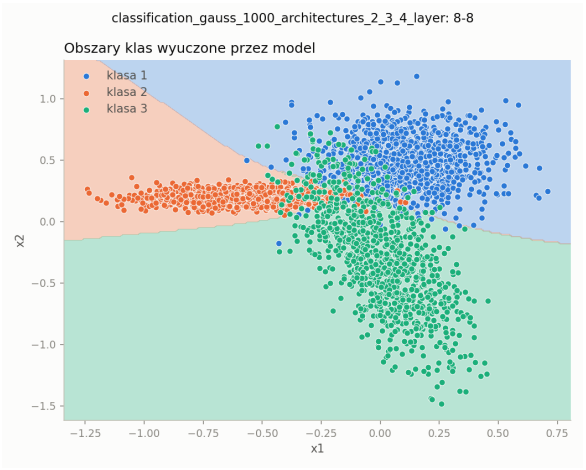
Rysunek 19 pokazuje, że sieci z co najmniej dwoma neuronami osiągają ok. 92–93% już po pierwszej epoce, a potem dokładność waha się o ok. ±0,5 punktu procentowego między epokami. Sieć z jednym neuronem przez pierwsze dwie epoki ma dokładność ok. 65–70% i nigdy nie przekracza 84%.

Dwie, trzy i cztery warstwy ukryte. Wyniki dla sieci głębszych przedstawia tabela 8.

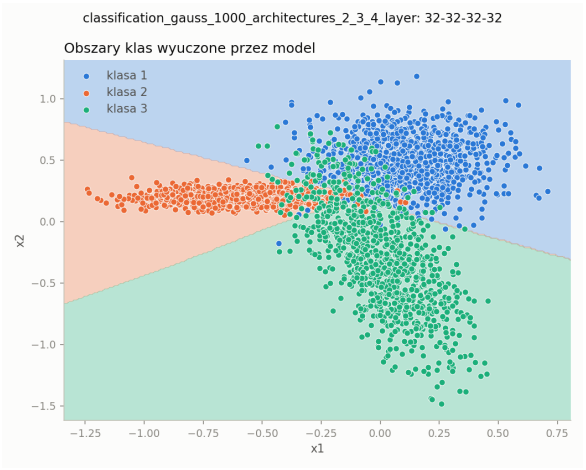
Tabela 8: Sieci z 2–4 warstwami ukrytymi na zbiorze `three_gauss_1000`, Sigmoid, 20 epok, ziarno 42.

Warstwy ukryte	Architektura	Parametry	Dokładność [%]
2	2-2-2-3	21	91,87
	2-8-8-3	123	**93,60**
	2-8-4-3	75	93,20
3	2-4-4-4-3	67	92,77
4	2-4-4-4-4-3	87	92,83
	2-32-32-32-32-3	3363	93,30

Wszystkie sieci głębokie osiągają 92–94%, czyli tyle samo co sieć z jedną warstwą ukrytą z dwoma neuronami. Dodatkowe warstwy nie poprawiają wyniku, bo problemem nie jest zbyt mała złożoność granicy decyzyjnej, tylko nakładanie się klas. Granice wyuczone przez sieć 2-8-8-3 i 2-32-32-32-32-3 są bardzo podobne (rysunek 20), mimo że druga sieć ma 27 razy więcej parametrów.



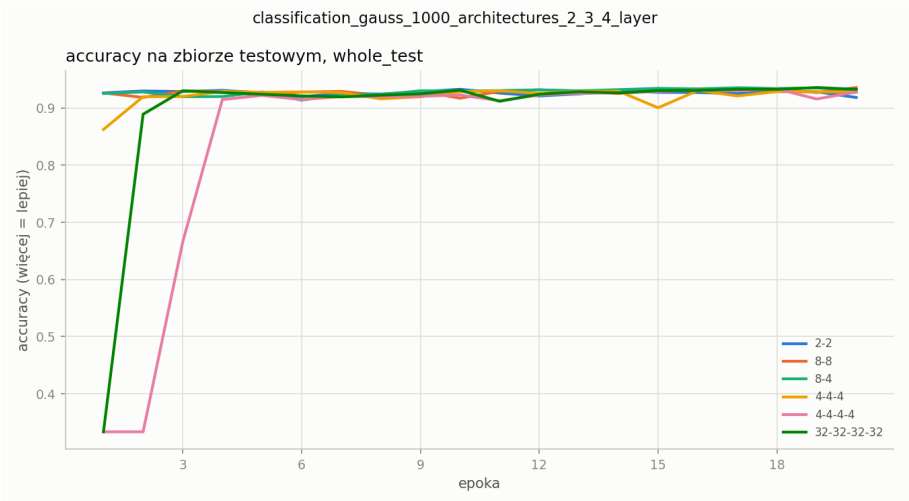
(a) 2-8-8-3 (93,6%)



(b) 2-32-32-32-32-3 (93,3%)

Rysunek 20: Obszary decyzyjne sieci z dwiema i czterema warstwami ukrytymi (`three_gauss_1000`).

Głębokość sieci wyraźnie wpływa natomiast na początek uczenia (rysunek 21). Sieci 2-4-4-4-4-3 i 2-32-32-32-32-3 przez pierwsze jedną lub dwie epoki mają dokładność 33%, czyli przypisują wszystkie punkty do jednej klasy, jak przy losowym zgadywaniu. Dopiero później „ruszają" i dochodzą do wyniku pozostałych sieci. Jest to skutek zanikającego gradientu: przy propagacji wstecznej gradient jest w każdej warstwie mnożony przez pochodną funkcji sigmoidalnej (nie większą niż 0,25), więc do pierwszych warstw głębokiej sieci dociera bardzo słaby sygnał [2].



Rysunek 21: Dokładność testowa w kolejnych epokach dla sieci z 2–4 warstwami ukrytymi (`three_gauss_1000`).

Seria z 300 epokami i trzema ziarnami. Tabela 9 pokazuje wyniki serii dodatkowej na mniejszym zbiorze `three_gauss_100` przy dłuższym uczeniu. Wnioski są takie same: jeden neuron ukryty nie wystarcza (82,6%), a od dwóch neuronów wzwyż oraz dla sieci z 0–4 warstwami ukrytymi dokładność mieści się w przedziale 92,8–93,6%.

Tabela 9: Seria dodatkowa: `three_gauss_100` (300 punktów testowych), Sigmoid, 300 epok. Średnia ± odchylenie standardowe z 3 ziaren.

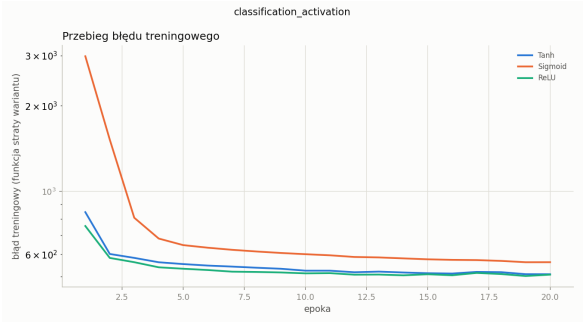
Architektura	Parametry	Dokładność [%]
2-3 (bez warstwy ukrytej)	9	$92{,}78 \pm 0{,}16$
2-1-3	9	$82{,}56 \pm 0{,}16$
2-2-3	15	$93{,}33 \pm 0{,}00$
2-8-3	51	$93{,}11 \pm 0{,}31$
2-16-3	99	$\mathbf{93{,}56 \pm 0{,}16}$
2-32-3	195	$93{,}11 \pm 0{,}16$
2-64-3	387	$93{,}11 \pm 0{,}16$
2-8-8-3	123	$\mathbf{93{,}56 \pm 0{,}42}$
2-8-8-8-3	195	$92{,}89 \pm 0{,}16$
2-8-8-8-8-3	267	$93{,}00 \pm 0{,}00$

6.5 Wpływ funkcji aktywacji

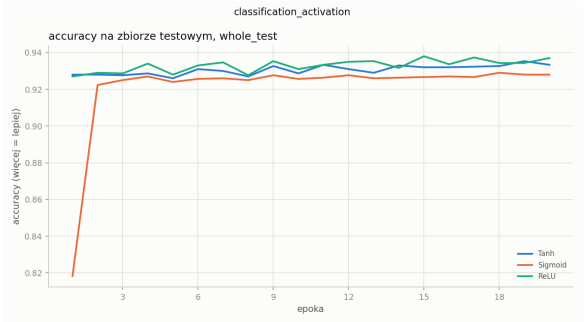
Porównano funkcje Tanh, Sigmoid i ReLU w warstwach ukrytych sieci 2-8-8-3 (tabela 10). Na wyjściu zawsze była funkcja Softmax, ponieważ wyjścia sieci mają być prawdopodobieństwami klas.

Tabela 10: Dokładność [%] na zbiorze testowym dla różnych funkcji aktywacji w warstwach ukrytych.

	`three_gauss_1000`, 2-8-8-3, 20 epok		`three_gauss_100`, 2-8-3, 300 epok
Aktywacja	$\eta = 0{,}1$ (ziarno 42)	$\eta = 0{,}01$ (ziarno 42)	$\eta = 0{,}1$ (3 ziarna)
Tanh	92,83	93,33	$92{,}67 \pm 0{,}47$
Sigmoid	**93,60**	92,80	$\mathbf{93{,}11 \pm 0{,}31}$
ReLU	92,70	**93,70**	$\mathbf{93{,}11 \pm 0{,}16}$



(a) Błąd uczenia (skala logarytmiczna)



(b) Dokładność testowa

Rysunek 22: Przebieg uczenia sieci 2-8-8-3 dla różnych funkcji aktywacji (`three_gauss_1000`, $\eta = 0{,}01$).

Końcowe dokładności różnią się o mniej niż 1 punkt procentowy, a kolejność funkcji zmienia się wraz z krokiem uczenia. Przy $\eta = 0{,}1$ najlepsza jest funkcja sigmoidalna, a przy $\eta = 0{,}01$ najgorsza. Nie można więc wskazać funkcji wyraźnie lepszej pod względem wyniku końcowego. Różnica jest widoczna w szybkości uczenia (rysunek 22): przy małym kroku sieć z funkcją sigmoidalną ma po pierwszej epoce dokładność tylko 82% (Tanh i ReLU ok. 93%), a jej błąd uczenia jest przez całe uczenie największy. Jest to ten sam efekt co w regresji: mała pochodna

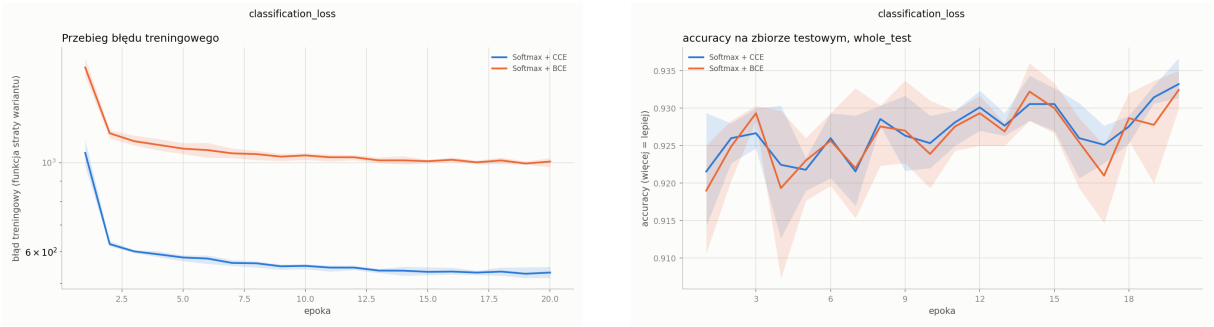
funkcji sigmoidalnej spowalnia uczenie [4], więc funkcja ta potrzebuje większego kroku. Przy kroku 0,1 ta przewaga Tanh i ReLU znika.

6.6 Wpływ funkcji błędu

Porównano entropię krzyżową (CCE), binarną entropię krzyżową (BCE) liczoną osobno dla każdego wyjścia oraz błąd średniokwadratowy (MSE), z wyjściem Softmax lub Sigmoid. Przy wyjściu sigmoidalnym każde wyjście jest niezależną oceną przynależności do jednej klasy i wyjścia nie sumują się do 1. Klasą przewidzianą pozostaje jednak wyjście o największej wartości. Wyniki przedstawia tabela 11.

Tabela 11: Dokładność [%] na zbiorze testowym dla różnych wyjść i funkcji błędu. Średnia ± odchylenie standardowe z 3 ziaren.

Wyjście + funkcja błędu	`three_gauss_1000` 2-8-8-3, 20 epok	`three_gauss_100` 2-8-3, 300 epok
Softmax + CCE	**93,32 ± 0,24**	93,11 ± 0,31
Softmax + BCE	93,24 ± 0,20	—
Softmax + MSE	—	93,33 ± 0,27
Sigmoid + MSE	—	**93,44 ± 0,31**
Sigmoid + BCE	—	93,11 ± 0,16



(a) Błąd uczenia (skala logarytmiczna)

(b) Dokładność testowa

Rysunek 23: Przebieg uczenia dla funkcji błędu CCE i BCE (`three_gauss_1000`, średnia z 3 ziaren, zacieniowany obszar – odchylenie standardowe).

Wszystkie warianty dają praktycznie tę samą dokładność: różnice między nimi (do 0,3 punktu procentowego) są mniejsze niż odchylenie standardowe między ziarnami. Przebiegi dokładności dla CCE i BCE w kolejnych epokach niemal się pokrywają (rysunek 23). Wartości błędu uczenia nie są ze sobą porównywalne, ponieważ BCE sumuje błędy po wszystkich trzech wyjściach, a CCE uwzględnia tylko wyjście poprawnej klasy. Dlatego błąd BCE jest ok. dwa razy większy, chociaż sieć klasyfikuje tak samo dobrze. Na tak prostym problemie wybór funkcji błędu nie ma więc znaczenia dla wyniku. Połączenie Softmax + CCE pozostaje jednak naturalnym wyborem dla klasyfikacji wieloklasowej, ponieważ wyjścia można interpretować jako prawdopodobieństwa klas, a gradient względem pobudzeń warstwy wyjściowej ma prostą postać $\hat{y} - y$ [3].

6.7 Powtarzalność wyników

W eksperymentach powtórzonych z trzema ziarnami odchylenie standardowe dokładności wynosiło od 0 do 0,5 punktu procentowego. Podobnej wielkości są wahania dokładności testowej między kolejnymi epokami jednego przebiegu (rysunki 19 i 23). Różnice mniejsze niż ok. 1 punkt

procentowy, np. między funkcjami aktywacji, funkcjami błędu czy architekturami z co najmniej dwoma neuronami ukrytymi, nie są więc istotne. Wiarygodne są natomiast duże i powtarzalne różnice: sieć z jednym neuronem ukrytym na zbiorze `three_gauss` jest o ok. 10 punktów procentowych gorsza w obu seriach i dla wszystkich ziaren.

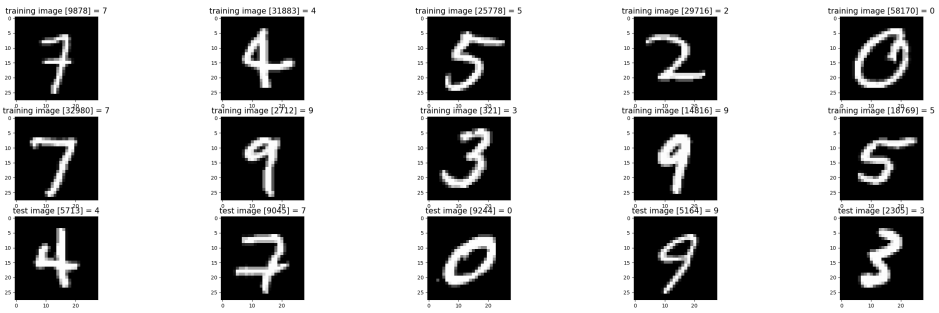
6.8 Podsumowanie wyników klasyfikacji

- Zbiór `simple` jest liniowo separowalny i wystarcza do niego sieć bez warstwy ukrytej (100% na `simple_100`, 99,6% na `simple_10000`). Większe sieci nie poprawiają wyniku.
- Na zbiorze `three_gauss` wszystkie sensowne architektury osiągają 92–94%. Górne ograniczenie wynika z nakładania się klas, a nie z możliwości sieci. Granice liniowe (sieć bez warstwy ukrytej) dają już ok. 92,5%.
- Wąskie gardło z jednym neuronem ukrytym wymusza równoległe granice decyzyjne i obniża dokładność do ok. 83%. Dwa neurony ukryte wystarczają.
- Głębokie sieci (4 warstwy ukryte z funkcją sigmoidalną) nie poprawiają wyniku i mają opóźniony początek uczenia z powodu zanikającego gradientu.
- Funkcja aktywacji ma niewielki wpływ na wynik końcowy (poniżej 1 punktu procentowego). Funkcja sigmoidalna uczy się wolniej przy małym kroku uczenia.
- Funkcje błędu CCE, BCE i MSE dają taką samą dokładność w granicach zmienności losowej.

7 Część druga – rozpoznawanie cyfr MNIST

7.1 Opis problemu i zbiór danych

Celem drugiej części projektu było nauczenie sieci zaimplementowanej w części pierwszej rozpoznawania ręcznie pisanych cyfr. Wykorzystano zbiór MNIST [5], który zawiera obrazy cyfr od 0 do 9 w skali szarości o rozmiarze 28×28 pikseli (wartości jasności od 0 do 255). Zbiór jest podzielony na 60 000 obrazów uczących i 10 000 obrazów testowych. Jest to zadanie klasyfikacji wieloklasowej z dziesięcioma klasami. Przykładowe obrazy przedstawiono na rysunku 24.



Rysunek 24: Przykładowe obrazy ze zbioru uczącego MNIST wraz z etykietami.

Przygotowanie danych było proste i nie wymagało rozszerzania implementacji:

- każdy obraz 28×28 został spłaszczony do wektora o długości 784,
- wartości pikseli podzielono przez 255, aby znalazły się w przedziale $[0, 1]$ (zbyt duże wartości wejściowe powodowałyby bardzo duże pobudzenia neuronów z funkcją ReLU),

- etykiety zakodowano w postaci *one-hot*, czyli jako wektory długości 10 z jedynką na pozycji odpowiadającej cyfrze.

Do uczenia używano wyłącznie zbioru uczącego. Zbiór testowy służył tylko do oceny sieci: po każdej epoce liczono na nim błąd i dokładność, aby pokazać przebieg uczenia na wykresach, ale wartości te nie miały żadnego wpływu na wagi sieci (nie stosowano np. wczesnego zatrzymania).

7.2 Architektura sieci i parametry uczenia

Zastosowano perceptron wielowarstwowy z części pierwszej bez zmian w algorytmie uczenia. Modyfikowano jedynie parametry sieci: liczbę i liczność warstw ukrytych, funkcję aktywacji, krok uczenia, rozmiar paczki (*batch size*) oraz sposób zmiany kroku uczenia między epokami. Elementy wspólne dla wszystkich przebiegów:

- warstwa wejściowa: 784 neurony (jeden na piksel),
- warstwa wyjściowa: 10 neuronów (jeden na cyfrę) z funkcją Softmax, dzięki czemu wyjścia sieci można interpretować jako prawdopodobieństwa przynależności do klas,
- funkcja błędu: entropia krzyżowa (CCE), naturalnie dopasowana do wyjścia Softmax,
- inicjalizacja wag z rozkładu $\mathcal{N}(0,\, 1/n_{\text{in}})$ [4],
- uczenie metodą SGD (dla paczki rozmiaru 1) lub mini-batch SGD (dla paczki rozmiaru 32), z tasowaniem danych w każdej epoce,
- 10 epok uczenia, ziarno generatora liczb losowych `seed = 42`.

Oprócz stałego kroku uczenia sprawdzono krok malejący wykładniczo (`ExponentialDecay`), w którym po każdej epoce krok jest mnożony przez 0,75, ale nie spada poniżej 10^{-4}:

$$\eta_e = \max\Big(\eta_0 \cdot 0{,}75^{\,e-1},\ 10^{-4}\Big), \qquad e = 1, 2, \ldots, 10, \tag{3}$$

gdzie η_0 to początkowy krok uczenia, a e to numer epoki. Dla $\eta_0 = 0{,}01$ krok maleje od 0,01 w pierwszej epoce do około $7{,}5 \cdot 10^{-4}$ w dziesiątej. Duży krok na początku pozwala szybko zbliżyć się do dobrego rozwiązania, a mniejszy pod koniec ogranicza „skakanie" wag wokół minimum, które przy SGD z paczką rozmiaru 1 jest szczególnie widoczne.

7.3 Przebieg eksperymentów

Eksperymenty prowadzono w kilku krokach, za każdym razem zmieniając jeden lub dwa parametry:

1. **Punkt wyjścia**: sieć 784-128-10 z funkcją Sigmoid w warstwie ukrytej, paczka rozmiaru 32. Sieć uczyła się poprawnie, ale wolno (tabela 12).
2. **Zmiana aktywacji na ReLU i paczki na 1**: sieć 784-128-10 z ReLU uczona czystym SGD. Sprawdzono stały krok 0,05 i 0,01 oraz krok malejący według wzoru (3).
3. **Większa sieć**: dodano drugą warstwę ukrytą, uzyskując architekturę 784-256-128-10, i powtórzono porównanie kroku stałego z malejącym.
4. **Powrót do paczki rozmiaru 32** dla architektury 784-256-128-10 z krokiem 0,01 i 0,1.
5. **Sprawdzenie na podzbiorze**: uczenie najlepszej konfiguracji tylko na pierwszych 500 obrazach, aby zobaczyć wpływ liczby danych uczących.

7.4 Wyniki

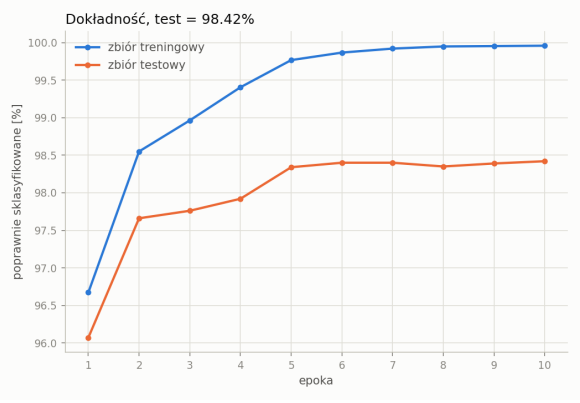
Zestawienie wszystkich pełnych przebiegów (10 epok, ziarno 42) przedstawia tabela 12.

Tabela 12: Wyniki uczenia na MNIST po 10 epokach (ziarno 42, wyjście Softmax + CCE). Dokładność podana dla zbioru uczącego i testowego. Krok „malejący” według wzoru (3).

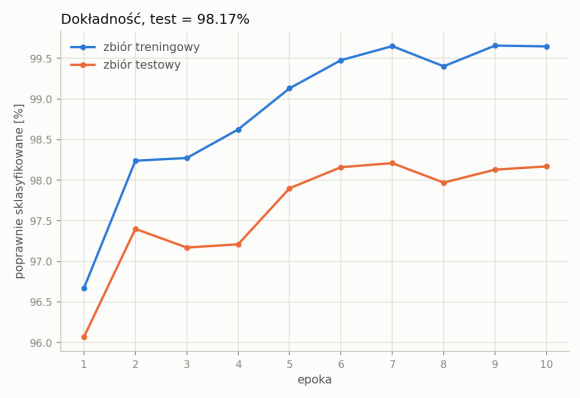
Architektura	Aktywacja	Batch	η_0	Krok	Dokładność [%]	
					uczenie	test
784-128-10	Sigmoid	32	0,1	stały	96,58	96,20
784-128-10	Sigmoid	32	0,01	stały	94,99	94,97
784-128-10	ReLU	1	0,05	stały	96,97	95,61
784-128-10	ReLU	1	0,01	stały	99,52	97,81
784-128-10	ReLU	1	0,01	malejący	99,71	98,08
784-256-128-10	ReLU	1	0,01	stały	99,65	98,17
784-256-128-10	ReLU	1	0,01	malejący	99,96	**98,42**
784-256-128-10	ReLU	32	0,01	stały	96,79	96,32
784-256-128-10	ReLU	32	0,1	stały	99,93	98,32
784-256-128-10	ReLU	32	0,1	malejący	99,51	97,92
784-256-128-10	ReLU	1	0,01	malejący*	99,80	83,85

* uczenie tylko na pierwszych 500 obrazach ze zbioru uczącego.

Wpływ zmiennego kroku uczenia. Rysunek 25 porównuje przebieg uczenia sieci 784-256-128-10 z krokiem malejącym i stałym. Przy kroku malejącym dokładność testowa rośnie najszybciej: już po piątej epoce osiąga 98,34%, a w kolejnych epokach praktycznie się nie zmienia (98,40–98,42%). Przy stałym kroku 0,01 przebieg jest wyraźnie bardziej nierówny: dokładność testowa w trzeciej i ósmej epoce spada, a błąd testowy po szóstej epoce zaczyna rosnąć (z 0,068 do 0,077). Końcowa dokładność testowa jest o 0,25 punktu procentowego niższa (98,17%). Ten sam efekt występuje dla mniejszej sieci 784-128-10: krok malejący poprawił wynik z 97,81% do 98,08%.



(a) Krok malejący: $\eta_0 = 0{,}01$, mnożony przez 0,75 co epokę.

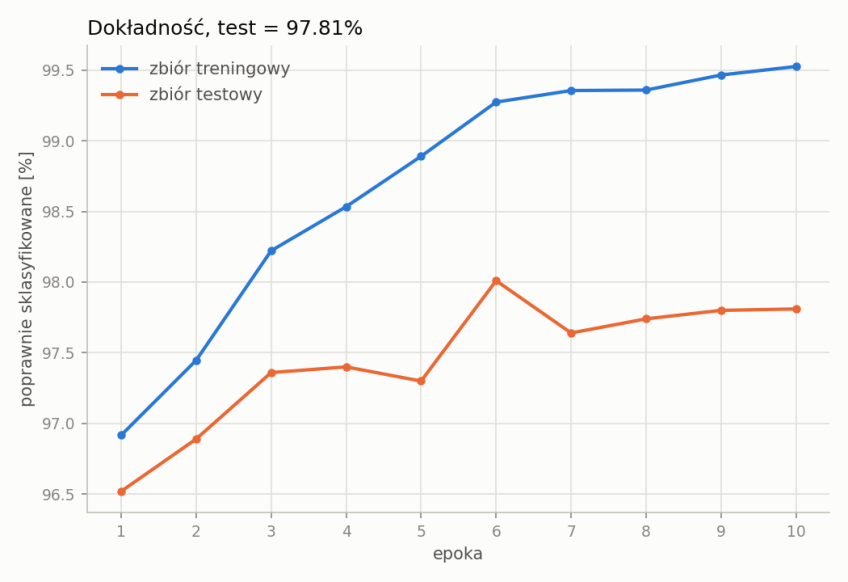


(b) Krok stały: $\eta = 0{,}01$.

Rysunek 25: Dokładność na zbiorze uczącym i testowym w kolejnych epokach dla sieci 784-256-128-10 (ReLU, paczka rozmiaru 1, ziarno 42).

Wpływ architektury. Dodanie drugiej warstwy ukrytej (784-256-128-10 zamiast 784-128-10) poprawiło dokładność testową zarówno przy kroku stałym (97,81% → 98,17%), jak i malejącym (98,08% → 98,42%). Przebieg uczenia mniejszej sieci przedstawia rysunek 26. Różnica między

dokładnością na zbiorze uczącym a testowym (ok. 1,5–1,7 punktu procentowego) jest podobna dla obu architektur, czyli większa sieć nie przeuczyła się wyraźnie bardziej w ciągu 10 epok.



Rysunek 26: Dokładność na zbiorze uczącym i testowym w kolejnych epokach dla sieci 784-128-10 (ReLU, paczka rozmiaru 1, $\eta = 0{,}01$, ziarno 42).

Wpływ funkcji aktywacji, kroku uczenia i rozmiaru paczki. Sieć z funkcją Sigmoid i paczką rozmiaru 32 uczyła się wolno: po 10 epokach osiągnęła 96,20% przy $\eta = 0{,}1$ i tylko 94,97% przy $\eta = 0{,}01$, a dokładność wciąż rosła. Przy paczce rozmiaru 32 wagi są aktualizowane 32 razy rzadziej niż przy paczce rozmiaru 1, dlatego przy tym samym kroku 0,01 sieć 784-256-128-10 osiągnęła tylko 96,32%. Zwiększenie kroku do 0,1 zrównoważyło mniejszą liczbę aktualizacji i dało 98,32%, czyli wynik zbliżony do najlepszego. Malejący krok przy paczce 32 pogorszył wynik (97,92%), prawdopodobnie dlatego, że krok zmniejszał się, zanim sieć zdążyła się w pełni nauczyć. Z kolei przy paczce rozmiaru 1 krok 0,05 okazał się zbyt duży: uczenie było niestabilne (w trzeciej epoce dokładność testowa spadła z 94,66% do 91,78%), a końcowy wynik wyniósł tylko 95,61%.

Wpływ liczby danych uczących. Ta sama konfiguracja co najlepsza, ale uczona tylko na 500 obrazach, osiągnęła 99,80% na zbiorze uczącym i jedynie 83,85% na testowym. Sieć zapamiętała przykłady uczące, ale nie nauczyła się uogólniać, co pokazuje, że do dobrego wyniku potrzebny jest cały zbiór uczący.

Powtarzalność. Przebiegi, które różnią się tylko sposobem zmiany kroku uczenia (stały lub malejący), mają identyczne wyniki po pierwszej epoce (np. 96,07% na zbiorze testowym dla obu wariantów sieci 784-256-128-10), ponieważ krok zaczyna maleć dopiero po pierwszej epoce. Potwierdza to, że przy tym samym ziarnie inicjalizacja wag i kolejność danych są identyczne, a uczenie jest w pełni powtarzalne.

7.5 Najlepsza sieć i odtworzenie uczenia

Najlepszy wynik, **98,42% poprawnie rozpoznanych cyfr ze zbioru testowego**, osiągnęła sieć o parametrach z tabeli 13.

Tabela 13: Parametry potrzebne do odtworzenia uczenia najlepszej sieci.

Parametr	Wartość
Architektura	784-256-128-10
Aktywacja w warstwach ukrytych	ReLU
Wyjście i funkcja błędu	Softmax + CCE
Inicjalizacja wag	$\mathcal{N}(0,\, 1/n_{\text{in}})$
Rozmiar paczki	1 (SGD)
Początkowy krok uczenia η_0	0,01
Zmiana kroku	`ExponentialDecay(rate=0.75, min_step=0.0001)`
Liczba epok	10
Ziarno generatora	42
Dane uczące	pełny zbiór uczący (60 000 obrazów)

Architekturę i sposób zmiany kroku ustawia się w pliku `training_mnist_model.py`:

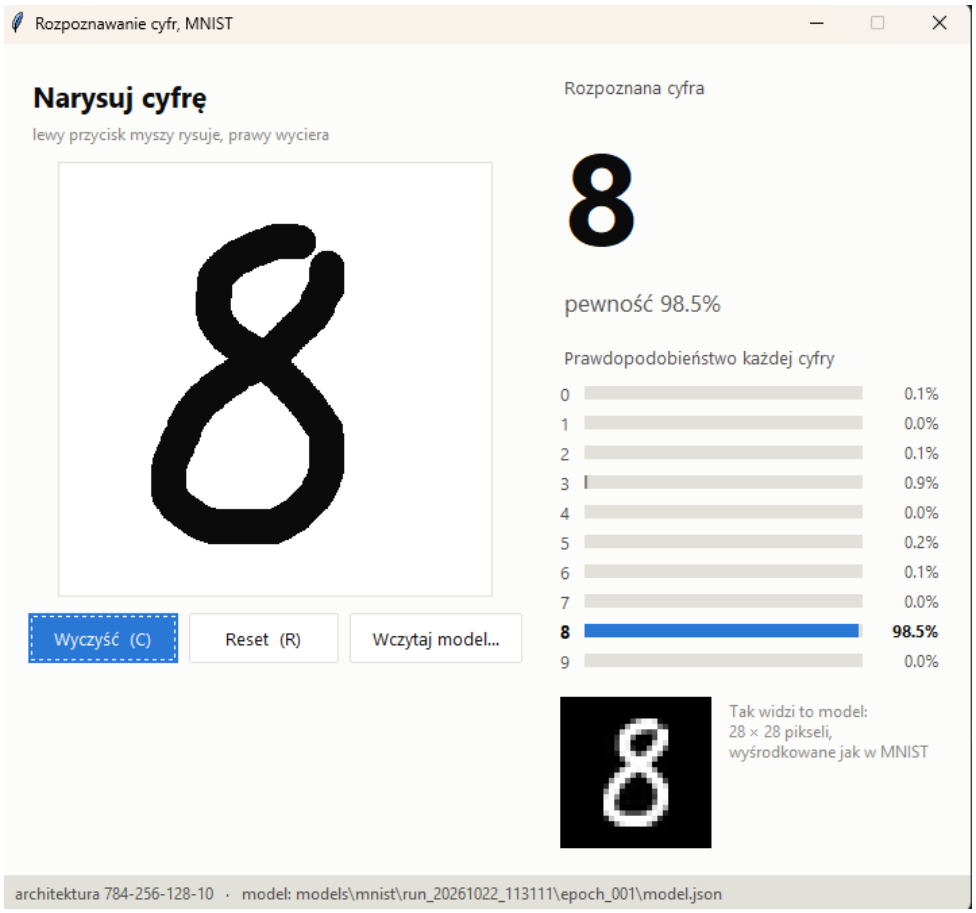
- stała `LAYERS = [784, 256, 128, 10]`,
- w funkcji `build_model` argument `learning_step_update=ExponentialDecay(rate=0.75, min_step=0.0001)`,

a pozostałe parametry podaje się przy uruchomieniu:

```
python training_mnist_model.py -epochs 10 -batch-size 1 -learning-rate 0.01 -seed 42
```

7.6 Weryfikacja na własnoręcznie narysowanych cyfrach

Oprócz oceny na zbiorze testowym przygotowano prosty program, w którym można narysować cyfrę myszką (plik `digit_recognizer.py`). Rysunek jest zamieniany na 784 wartości z przedziału $[0, 1]$ i podawany na wejście nauczonej sieci, a program wyświetla prawdopodobieństwa wszystkich dziesięciu klas (rysunek 27). Jest to jedynie sprawdzenie jakościowe, ale pokazuje, że sieć radzi sobie także z cyframi spoza zbioru MNIST, napisanymi przez inną osobę i innym „narzędziem”.



Rysunek 27: Program do ręcznego rysowania cyfr: narysowana cyfra i prawdopodobieństwa klas zwrócone przez najlepszą sieć.

7.7 Wnioski z części drugiej

- Implementacja z części pierwszej, bez dodawania nowych funkcji, pozwoliła osiągnąć 98,42% dokładności na zbiorze testowym MNIST.
- Najlepiej sprawdziła się sieć 784-256-128-10 z ReLU, uczona SGD (paczka rozmiaru 1) z krokiem malejącym od 0,01. Uczyła się najszybciej i już po piątej epoce osiągnęła wynik bliski końcowemu.
- Malejący krok uczenia dawał wyższą i stabilniejszą dokładność niż krok stały przy paczce rozmiaru 1, dla obu badanych architektur.
- Druga warstwa ukryta poprawiła wynik o ok. 0,3–0,4 punktu procentowego.
- ReLU w warstwach ukrytych uczyła się wyraźnie szybciej niż Sigmoid.
- Przy paczce rozmiaru 32 wymagany był większy krok (0,1). Wtedy wynik (98,32%) był prawie tak dobry jak najlepszy, a uczenie trwało krócej.
- Mała liczba danych uczących (500 obrazów) prowadzi do silnego przeuczenia: 99,8% na zbiorze uczącym, ale tylko 83,9% na testowym.

8 Podsumowanie i wnioski

W ramach projektu zaimplementowano od podstaw perceptron wielowarstwowy z propagacją wsteczną błędu, SGD, konfigurowalną architekturą, kilkoma funkcjami aktywacji i błędu oraz powtarzalnym (opartym na ziarnie) procesem uczenia. Zaimplementowano też wizualizacje wyników, przebiegu błędu i ewolucji wag. Implementację sprawdzono w zadaniach regresji i klasyfikacji na zbiorach syntetycznych, a następnie bez rozszerzania jej o nowe funkcje zastosowano do zbioru MNIST, uzyskując 98,42% dokładności na zbiorze testowym. Najważniejsze wnioski zebrano poniżej według badanych czynników.

Funkcje aktywacji

- **Funkcja sigmoidalna uczy się najwolniej we wszystkich zadaniach.** W regresji jej końcowy błąd uczenia był ok. 8 razy większy niż dla Tanh (tabela 2). W klasyfikacji przy małym kroku uczenia osiągała po pierwszej epoce 82% wobec 93% dla Tanh i ReLU. Wynika to z małej pochodnej (maksymalnie 0,25) i braku wyśrodkowania wokół zera. Funkcja sigmoidalna potrzebuje większego kroku uczenia, a przy kroku 0,1 jej przewaga konkurentów w klasyfikacji znikała.
- **Tanh dał najlepsze wyniki w regresji**: najszybszy spadek błędu, najmniejsze MSE i gładkie dopasowanie funkcji.
- **ReLU daje funkcję kawałkami liniową.** W regresji widać to jako „załamania" predykcji i liniowe przedłużenie funkcji poza zakresem uczącym, a w klasyfikacji jako proste granice decyzyjne. W sieci MNIST ReLU pozwoliło szybko uczyć sieci z wieloma neuronami.
- W klasyfikacji na zbiorze `three_gauss` różnice między funkcjami aktywacji w wyniku końcowym nie przekraczały 1 punktu procentowego, a ich kolejność zmieniała się wraz z krokiem uczenia (tabela 10). Krok uczenia miał tu większy wpływ niż sama funkcja aktywacji.

Liczba warstw ukrytych i neuronów

- **Bez warstwy ukrytej sieć jest modelem liniowym**: regresją liniową (dla wyjścia liniowego) albo regresją logistyczną (dla wyjścia Softmax). Wystarcza to dla liniowo separowalnego zbioru `simple` (100%), a nawet daje ok. 92,5% na `three_gauss`. W regresji funkcji nieliniowych błąd był jednak ok. 170 razy większy niż dla sieci z warstwą ukrytą.
- **Szerokość warstw jest ważniejsza niż ich liczba.** Jeden neuron ukryty (wąskie gardło) wymusza równoległe granice decyzyjne i obniża dokładność na `three_gauss` do ok. 83%. Głębokie sieci z warstwami o 1–2 neuronach uczyły się w regresji tak źle jak sieć z jednym neuronem. Natomiast już dwa neurony w klasyfikacji i 16–64 w regresji wystarczały do osiągnięcia najlepszych wyników.
- **Dodatkowe warstwy nie pomogły w prostych problemach**, a w sieciach z funkcją sigmoidalną lub Tanh utrudniały początek uczenia. Sieci z czterema warstwami ukrytymi przez pierwsze epoki klasyfikowały wszystkie punkty do jednej klasy (rysunek 21), co jest objawem zanikającego gradientu. Druga warstwa ukryta przydała się dopiero w trudniejszym zadaniu MNIST (poprawa o ok. 0,35 punktu procentowego), czyli potrzebna głębokość zależy od złożoności problemu.

Funkcja błędu i aktywacja wyjścia

- **Aktywacja wyjścia musi pasować do zadania.** W regresji wyjście sigmoidalne ogranicza predykcje do przedziału $(0, 1)$, przez co sieć nie może przewidzieć wartości spoza zakresu zbioru uczącego. Błąd na całym zbiorze testowym był ponad dwa razy większy niż dla wyjścia liniowego (tabela 5). W klasyfikacji naturalnym wyborem jest Softmax, którego wyjścia można interpretować jako prawdopodobieństwa klas.
- **Przy wyjściu sigmoidalnym BCE uczy szybciej niż MSE**: w regresji błąd na zakresie uczącym był 3,5 razy mniejszy, ponieważ w gradiencie BCE skraca się mała pochodna funkcji sigmoidalnej.
- **W klasyfikacji wybór funkcji błędu nie wpłynął na wynik.** CCE, BCE i MSE dawały dokładność różniącą się o mniej niż odchylenie standardowe między ziarnami (tabela 11).
- Wartości różnych funkcji błędu nie są ze sobą porównywalne (np. BCE nie osiąga zera dla celów z wnętrza $(0, 1)$ i sumuje błąd po wszystkich wyjściach), dlatego warianty porównywano wspólną miarą: MSE w regresji i dokładnością w klasyfikacji.

Ograniczenia wynikające z danych

- **Sieć nie ekstrapoluje.** Poza zakresem danych uczących jej zachowanie wynika z kształtu funkcji aktywacji (wypłaszczenie dla Tanh i Sigmoid, linia prosta dla ReLU), a nie z danych. Dlatego żadna sieć nie odtworzyła szybkiego wzrostu funkcji sześciennej. Funkcję `activation`, która wypłaszcza się tak samo jak Tanh, ekstrapolowano natomiast prawie bezbłędnie.
- **Nakładanie się klas wyznacza górną granicę dokładności.** Na `three_gauss` wszystkie sensowne architektury zatrzymywały się na 92–94%, a dalsze powiększanie sieci tego nie zmieniało.
- **Mało danych prowadzi do przeuczenia.** Sieć MNIST uczona na 500 obrazach osiągnęła 99,8% na zbiorze uczącym, ale tylko 83,9% na testowym.

Uczenie sieci MNIST

Najlepszy wynik (98,42%) dała sieć 784-256-128-10 z ReLU, uczona SGD z krokiem malejącym od 0,01. Malejący krok uczenia poprawił wynik dla obu badanych architektur i ustabilizował uczenie: dokładność bliską końcowej sieć osiągnęła już po pięciu epokach. Przy paczce rozmiaru 32 konieczne było zwiększenie kroku do 0,1, aby zrównoważyć mniejszą liczbę aktualizacji wag. Wynik był wtedy niewiele gorszy (98,32%).

Powtarzalność

Uczenie jest w pełni powtarzalne: przy tym samym ziarnie przebiegi są identyczne. Różne ziarna dają wyniki różniące się o 0–0,5 punktu procentowego w klasyfikacji, a w regresji bardzo małe błędy na zakresie uczącym zmieniają się nawet dwukrotnie (rozdziały 5.7 i 6.7). Za istotne uznano więc tylko różnice wyraźnie większe od tej zmienności, np. wpływ wąskiego gardła czy wyjścia sigmoidalnego w regresji. Różnic rzędu ułamków punktu procentowego nie należy interpretować jako przewagi jednej konfiguracji nad drugą.

Ograniczenia badań i możliwe rozszerzenia

Większość eksperymentów wykonano z jednym ziarnem, a uczenie klasyfikacji trwało tylko 20 epok. Powtórzenie wszystkich porównań dla kilku ziaren pozwoliłoby jednoznacznie rozstrzygnąć małe różnice. Konfigurację sieci MNIST wybierano na podstawie wyników na zbiorze testowym. Poprawniej byłoby wydzielić ze zbioru uczącego zbiór walidacyjny i zbiór testowy wykorzystać tylko raz, do oceny końcowej sieci. Naturalnymi rozszerzeniami są też metody przeciwdziałające przeuczeniu (regularyzacja, wczesne zatrzymanie) oraz warianty SGD z momentem, które mogłyby przyspieszyć uczenie głębszych sieci.

Literatura

[1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[2] I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.

[3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[4] Y. LeCun, L. Bottou, G. B. Orr, K.-R. Müller. Efficient BackProp. W: *Neural Networks: Tricks of the Trade*, LNCS 7700, Springer, 2012, s. 9–48.

[5] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[6] C. R. Harris i in. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[7] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.